ChatNPC

By

Ethan Heavey

Submitted in partial fulfillment of the
requirements for the degree of
Masters of Science in Computer Science
at
Saint Francis Xavier University
Antigonish, Nova Scotia
May 2024

© Copyright by Ethan Heavey, 2024

Saint Francis Xavier University
Department of
Mathematics, Statistics and Computer Science

The undersigned hereby certify that they have read a thesis entitled "**ChatNPC**" by **Ethan Heavey** in partial fulfillment of the requirements for the degree of **Masters of Science**.

Dated: May 31, 2024

Co-supervisor: _____
Dr. James Hughes

Co-supervisor: _____
Dr. Milton King

Committee Member: _____
Dr. Jean-Alexis Delamer

St. Francis Xavier University

May 2024

Author:	Ethan Heavey
Title:	ChatNPC
Department:	Mathematics, Statistics and Computer Science
Faculty:	Science
Convocation	May 2024

Permission is herewith granted to Saint Francis Xavier University to circulate and to have copied for non-commercial purposes, at its discretion, the above title upon request of individuals or institutions.

Ethan Heavey

The author reserves other publication rights, and neither the thesis nor extensive extractions from it may be printed or otherwise reproduced without the author's written permission.

The Author attests that permissions has been obtained for the use of any copyrighted material appearing in this thesis (other than brief excerpts requiring only proper acknowledgement in scholarly writing) and that all such use is clearly acknowledged.

To Mum and Dad

Abstract

Modern advancements in generative text models are revolutionizing how humans interact with technology and improve productivity. This Masters of Science thesis explores the idea of a dialogue system enhanced by said generative text models. A background on natural language processing is provided to ensure the terms and ideas are understood. My work on the natural language inference task and how it ties into a dialogue system is also discussed. Leveraging an external database, a contradiction detection system, a fact rephrasal system, and a unique approach to ranking similar records, ChatNPC allows users to have conversations with fictitious characters through a website — whilst allowing said characters to retain information from previous conversations.

Acknowledgements

Thank you to my supervisors and committee for their support.

Contents

1 A Tale of Two Concepts: Introduction **1**

2 The Fellowship of Findings: Related Work **3**

2.1 Terminology 4

2.2 Language Modelling 5

2.2.1 N-Gram 5

2.2.2 Neural Language Models 9

2.3 Embeddings 13

2.4 Generative Models 15

2.4.1 Applications of Generative Text 20

2.5 Prompt Engineering 21

2.6 Long Term Memory 22

2.7 Contradiction Detection and Natural Language Inference 23

2.8 Persistent Homology 25

3 The Wizard Behind the Project's Curtain: Data **29**

3.1 Combating Memory Limitations 29

3.1.1 Characters 30

3.1.2 Information Storage 30
3.1.3 Information Retrieval 35
3.2 Datasets 38
3.2.1 Sentences Involving Compositional Knowledge 38
3.2.2 Stanford Natural Language Inference 40
3.2.3 Multi-Genre Natural Language Inference 40

4 Unmasking Jekyll and Hyde: The Art of Contradiction Detection 45
4.1 Context Pool Saturation 45
4.2 Contradiction Detection Models 46
4.2.1 Natural Language Inference 46
4.2.2 Architecture 47
4.2.3 Vanilla Model Architecture 47
4.2.4 Augmented Model Architecture 49
4.2.5 Enhancements 52
4.3 Results 54
4.3.1 Vanilla Models 54
4.3.2 Augmented Models 55
4.4 The Verdict 61

5 Quality Control: Evaluation 65
5.1 Information Sent and Received 66
5.2 Response Time 67
5.3 Analysis 68

6 From World 1 to 8: Current Pipeline **70**

6.1 Username . . . 70

6.2 Character Selection . . . 73

6.2.1 List of Characters . . . 75

6.3 User Query . . . 75

6.4 Query Knowledge Base . . . 75

6.5 Contradiction Detection . . . 76

6.6 Prompt Engineering . . . 77

6.6.1 Alternate Templates . . . 80

6.7 Language Model . . . 81

6.7.1 Tested Language Models . . . 81

6.8 Text-to-Speech . . . 83

6.9 Output . . . 84

7 Curtain Call: Conclusion **86**

7.1 Great Scott! Future Work . . . 87

7.1.1 Evolving Environment . . . 87

7.1.2 Performance Trade-Offs . . . 88

7.1.3 Elevating the Interface . . . 88

7.1.4 Contradiction Detection System Improvements . . . 89

7.1.5 Character Background Information . . . 90

7.1.6 Town Background Information . . . 94

Bibliography **95**

List of Tables

2.1 A tokenized version of the sentence "the llama couldn't resist trying the lemonade.", where different tokens separated by commas. 5

2.2 The different word types and their counts from the example sentence "the llama couldn't resist trying the lemonade." 6

2.3 The vocabulary of a unigram model trained on the corpora "The llama couldn't resist trying the lemonade" and the number of occurrences of each token in the vocabulary. 8

2.4 The vocabulary of a bigram model trained on the corpora "The llama couldn't resist trying the lemonade" and the number of occurrences of each token in the vocabulary. 8

2.5 A term-term matrix for the phrase: "The llama couldn't resist trying the lemonade" 15

2.6 The top k tokens (proposed by a language model) and their probabilities (computed by the model, rounded to two digits), sorted in descending order by probability. These tokens will be used by the generative model to continue the input "salt and". 17

3.1 Examples of sentence pairs from the SICK dataset with their gold labels. 39

3.2 Examples of sentence pairs from the SNLI dataset with their gold labels. 41

3.3 Examples of sentence pairs from the MultiNLI dataset with their gold labels. 43

3.4 This table displays the class splits between the different datasets, post-cleaning, used to train and validate my models. 44

4.1 My scores for each model assessed in the experiments on the MultiNLI dataset. All scores are the results achieved by my models on both the M.Test and Mm.Test sets. The best results in each column are bolded. The BERT+TDA model from [Wu et al., 2022] is the baseline I'm comparing to. 56

4.2 Baseline scores for each LLM model assessed in the experiments on the SNLI dataset. All scores are the average of 30 iterations. The best results in each column are shown in bold. The baseline I'm comparing to is that established by the Entailment as Few-shot Learner (EFL) technique [Wang et al., 2021]. 57

4.3 Scores for each LLM model assessed in the experiments on the SICK dataset. All scores are the average of 30 iterations. The best results in each column are shown in bold. The baseline I'm comparing to is the CNN:CWE* + shallow features model presented by [Li et al., 2017]. . . . 58

4.4 Results achieved by each of my augmented models on M.Test and Mm.Test. The vanilla model results are shown in italics, and the baseline is presented at the top. The best results in each column are shown in bold. 62

4.5 The results achieved by each of my models on the SICK dataset. The results from the vanilla models are shown in italics, and the baseline is shown at the top. The best results in each column are shown in bold. . . 63

4.6 Results achieved by each of my models on the SNLI dataset. The vanilla model results are shown in italics, and the baseline is presented at the top. The best results in each column are shown in bold. 64

5.1 The questions using in quantitative experiments of ChatNPC 66

5.2 Average megabytes received per token during a conversation with ChatNPC. The lowest value is presented in bold. 67

5.3 Average megabytes sent MB per token during a conversation with ChatNPC. The lowest value is presented in bold. 67

5.4 The time, in seconds, ChatNPC takes to reply per token in the provided reply. The lowest value is presented in bold. 69

6.1 How the knowledge base is updated depending on the option chosen by the user. 78

List of Figures

2.1 The transformer model architecture [Vaswani et al., 2017] 12

2.2 An example of ChatGPT, a large language model, hallucinating. 19

2.3 PH features and their construction [Huang et al., 2018]. (a) Construction of *k-simplices* — fully-connected sets of $k+1$ data points. (b) Scatterplot of example data points. (c) Using a distance metric to quantify the distance between points, ϵ, all data points within ϵ of each other receive an edge between them. (d) The final set of features is formed as a complex — set of simplices. The coloured regions represent different features within the complex. 27

2.4 An example persistence diagram. The ϵ value on the $x - axis$ details the birth and death values of each feature. Dimension 0 features are displayed in the upper graph, whereas dimension 1 features are displayed in the lower one. 28

3.1 A visualization of a record as it is stored in the Pinecone database. . . . 34

6.1 The flow chart of the ChatNPC system. 72

6.2 The landing page for ChatNPC, where a user inputs a username to enable isolated character memories and interactions. 73

6.3 After selecting a username, ChatNPC brings you to the character selection screen. 74

6.4 The state of ChatNPC after choosing a character to converse with. Note: you are able to type in the text box at the bottom of the screen, and the pop-up telling you to select a character has disappeared. 74

6.5 A dialogue between myself — username penguins — with Melinda Deek, a fictitious character created for ChatNPC. When asked "What do you do?", Melinda replied with "I serve the village of Ashbourne as its knight, upholding the values of chivalry and protecting the people from threats such as bandits. My duties involve sword fighting and horsemanship, skills in which I have been trained since a young age. I reside in Redwood Manor, which is situated near the heart of Ashbourne." 76

6.6 After selecting a username, ChatNPC brings you to the character selection screen. 77

6.7 The selected option changes to a different colour, and Melinda then replies to our initial prompt with the updated context. 78

Chapter 1

A Tale of Two Concepts:

Introduction

Throughout history, the allure of both stories and games has held humanity firmly in its grip. As technological progress marches forward, our capacity to blur the boundaries between narrative and interaction has birthed a rich tapestry of innovative experiences. These experiences enable us to engage with stories one line, one scene, one image, or one input at a time, immersing ourselves in captivating worlds and narratives.

At the nexus of these two passions lies the realm of video games, where interactive gameplay and immersive storytelling unite to craft unique and unforgettable experiences for virtually everyone who partakes in them. In this modern era, digital landscapes beckon with their immersive vistas and intricate tales, presenting us with a tantalizing opportunity — a chance to redefine our relationship with one of the gaming world's foundational components: Non-Player Characters (NPCs).

NPCs are the unsung heroes of many gaming universes, yet they often exist as static, scripted entities, with predictable responses and often one-dimensional personalities. We

can, and should, do more to diversify these agents. Creating a dialogue system where NPCs are not just tools for advancing quests, but are dynamic, multifaceted characters that can enrich our gaming and immersive storytelling experiences in unique ways.

I have built a dialogue system — ChatNPC — that leverages natural language processing techniques to provide an immersive experience for its users. Different subtasks within the natural language processing field are tackled by ChatNPC, ranging from natural language inference, to an external knowledge base for large language models, to breaking up statements into their individual facts.

Chapter 2

The Fellowship of Findings: Related Work

The groundbreaking artificial intelligence (AI) language model from OpenAI, ChatGPT[1], has shed light on the power and diversity that large language models (LLMs) are capable of. These revolutionary models operate in a subdomain of AI called "Natural Language Processing" (NLP), which is an area of study in computer science that leverages the strengths of AI in order to assist computers in understanding spoken word and text in the same manner as humans. NLP combines deep learning, machine learning (ML), and statistical computer science models with linguistics, an area of study that analyzes language and its structure. To understand how AI models can be applied to the NLP tasks of natural language inference — determining if two statements have an entailed, contradictory, or neutral relationship — and text generation — the process of creating written content through algorithms of machine learning models —, we first need to understand how language models understand human language — through

[1] `https://openai.com/blog/chatgpt`

embeddings. We then have to understand how these embeddings are used to generate natural language, and how to prompt these language models to get the desired output.

2.1 Terminology

To ensure the rest of this related works section, which focuses primarily on natural language processing (NLP), makes the most sense, I'm going to briefly define a few recurring NLP terms.

A **token** is a unit of text that has been extracted from a larger body of text, typically through a process called "tokenization". As the building blocks for language modeling and may other NLP tasks, tokens are an essential component of NLP [Jurafsky, 2023]. While the output of the tokenization process varies based on the tokenization model used, a straightforward tokenization approach assigns one token to each word, defined by whitespace boundaries.[2]. An example of how that simple technique tokenizes the phrase "the llama couldn't resist trying the lemonade" is shown in Table 2.1. Notice how punctuation is given its own token!

Word **types** are distinct and unique forms of a word. Types represent the class of all tokens that have the same sequence of characters. Table 2.2 shows example of the different word types and their counts from the sentence "the llama couldn't resist trying the lemonade".

A **corpus** is a large and structured sets of text used for linguistic analysis, machine learning, and other language related studies. Corpora can be domain-specific (i.e. medical texts, legal documents, etc.) or more general (a collection of books).

[2]A more sophisticated tokenizer can be found at `https://platform.openai.com/tokenizer` if you want to check that out

Original	the llama couldn't resist trying the lemonade.
Tokens	the, llama, couldn't, resist, trying, the, lemonade, .

Table 2.1: A tokenized version of the sentence "the llama couldn't resist trying the lemonade.", where different tokens separated by commas.

In NLP, the **vocabulary** consists of all the unique word types present in a given corpus or dataset — a comprehensive list of the terms a language model has been trained on. A larger vocabulary allows language models to understand and work with a wider array of tokens. Vocabulary are, in summary, a smaller subset of the corpora a language model was trained on, with the size and diversity of said corpora impacting the richness and coverage of the vocaulary.

2.2 Language Modelling

Language models (LMs) are simply models that assign a probability to a sequence of words [Jurafsky, 2023]. The manner in which these models assign probabilities varies by model. In this section, we will discuss the simple n-gram LM and more complex neural network-based LMs.

2.2.1 N-Gram

The n-gram is one of the simplest types of language models. It finds a set of tokens within the text it was trained on, counts the frequency at which those sets of tokens occur, and employs said frequencies to predict the next token in a given phrase. The n

Word Type	Count
the	2
llama	1
couldn't	1
resist	1
trying	1
lemonade	1
.	1

Table 2.2: The different word types and their counts from the example sentence "the llama couldn't resist trying the lemonade."

comes into play when analyzing how much context we wish to give the model in order to make its prediction. A 5-gram model will consider the 4 most recent tokens when making its prediction as to what the next token should be. To generalize, an n-gram model will consider the most recent $n-1$ tokens from the input phrase, also referred to as a "prompt" throughout this paper, when calculating its prediction for the nth token.

Starting small, a unigram (where $n = 1$) is merely the total count of each token in the entire training data. A bigram (where $n = 2$) is the count of every pair of tokens in the model's training data, and so on and so forth. As an example, let's examine Tables 2.3 and 2.4, and the sentence:

The llama couldn't resist trying the lemonade.

The n-gram LM works to predict the next token in a sequence. Whilst all values of n will have the same vocabulary, they each observe different contexts. For example, the scope of the unigram model's context is a single token — rendering the model unable to capture context and token order. As simple LMs, unigrams do not consider the relationships between tokens, as these models look at individual tokens and not multiple in sequence. Bigram models extend their context window from zero to one token, and thus are able to interpret context surrounding certain tokens. If a bigram only had the vocabulary outlined in Table 2.4 and was asked to predict the next token in the sentence "After the banquet the", the bigram would have a 50% chance of choosing the "llama" token and a 50% chance of choosing the "lemonade" token. Those two tokens are the only two tokens in the bigram's vocabulary that appear after "the", thus a 50% chance for each to be chosen.

The reliance on using the model's vocabulary to calculate the probability of the next token in the sentence can present problems when there are tokens presented in

Vocabulary	the	llama	couldn't	resist	trying	lemonade
Count	2	1	1	1	1	1

Table 2.3: The vocabulary of a unigram model trained on the corpora "The llama couldn't resist trying the lemonade" and the number of occurrences of each token in the vocabulary.

Vocabulary	the llama	llama couldn't	couldn't resist	resist trying	trying the	the lemonade
Count	1	1	1	1	1	1

Table 2.4: The vocabulary of a bigram model trained on the corpora "The llama couldn't resist trying the lemonade" and the number of occurrences of each token in the vocabulary.

the input to the model that are *not* found in said model's vocabulary. For example, if we considered a trigram model (trained only on the same sentence as the previous unigram and bigram models) trying to predict the next token in the banquet sentence, the token "banquet" does not appear in its vocabulary. A workaround for this issue is to turn unseen tokens into the `UNK` token, representing an "unknown" token. This method allows the model to continue interpreting the text, losing some of the context in the process.

2.2.2 Neural Language Models

Neural networks have transformed Natural Language Processing (NLP). They surpass traditional n-gram models by capturing complex linguistic structures and semantic relationships through vast data training.

Feed-Forward Neural Networks

LMs with a feed-forward neural network architecture are modeled on the human brain, consisting of multiple layers in which the units (neurons) do not form cycles; the outputs from one layer are passed to the next one in sequence — never backwards to lower layers. Compared to n-grams, this architecture allows it to handle longer histories, generalize better over contextually similar tokens, and more accurately predict the next token in a sequence [Jurafsky, 2023]. Feed-forward neural language models also apply probabilities to tokens when trying to predict the next token in a sequence, albeit in a slightly more complicated process than that of the n-gram. These language models are trained on the task of forward inference; given an input, produce a probability distribution over possible outputs. To generate this distribution, the LM one-hot encodes[3] the input tokens, computes an embedding (by multiplying each input by an embedding matrix) for each input token, multiplies the embeddings by a weight matrix, and then applies an activation function element-wise to produce the hidden layer in the network. This hidden layer is then multiplied by another weight matrix, producing the probabilities for each potential token to continue the input. Using a softmax function, the network will choose the token with the highest probability to continue the input — the textual data

[3]A machine learning technique that allows categorical variables with no ordinal relationships to be represented by a sequence of 0s and 1s.

the network is analyzing to generate predictions about subsequent tokens. The skip-gram and continuous bag of words (CBOW) models [Mikolov et al., 2013] are examples of feed-forward neural language models.

Recurrent Neural Networks

A recurrent neural network (RNN) is a neural network that does contain cycles; future layers pass output back to previous ones. Compared to a feed-forward neural language model, RNNs are designed to handle sequential data, with network weights recalculated with each iteration and influenced by the value of the hidden layer from the previous time step. As language models, RNNs process the input sequence one token at a time and attempt to predict the next token based on the current token and the previous hidden state. The hidden state leveraged by this architecture allows RNNs to avoid the limited context problem present in n-grams by essentially letting the hidden layer represent information about all previously seen text. Generating a distribution of possible next tokens is performed near-identically to the feed-forward network approach, with a slight twist to incorporate the previous iteration's hidden state. Instead of generating a hidden layer from the input, RNNs take the previous iterations hidden layer, h_{t-1}, multiply it by a weight matrix, U, and add the weighted hidden state matrix to the matrix calculated by multiplying the one-hot encoded input x_t by a different weight matrix, W. This addition results in the hidden state for the current time step, h_t, which is subsequently passed through another weight matrix, V, yielding the distribution, y_t, as output. Using a softmax function, we can get the most likely token based on the input. Examples of RNN-based LMs are Context2Vec [Melamud et al., 2016] and Doc2Vec [Le and Mikolov, 2014], the former of which also uses a bidirectional technique to understanding input by

reading from left to right and right to left — providing context both before and after the input sequence to the LM.

Transformers (not the Hasbro kind)

The state-of-the-art neural model that is enjoying widespread success across the NLP field is the transformer architecture [Vaswani et al., 2017]. This architecture leverages "self-attention" to efficiently process sequential data by capturing long-range dependencies and contextual information. This mechanism allows a model to focus on different parts of the input sequence of tokens to varying degrees, capturing different dependencies and relationships between tokens. Transformers enable superior performance in various tasks such as machine translation [Gao et al., 2021], text generation [Brown et al., 2020], and sentiment analysis [Mishev et al., 2020]. The importance of the architecture lies in its ability to handle diverse natural language tasks and to train in parallel, making it a versatile and powerful tool. An example of the architecture can be seen in Figure 2.1.

Due to the hardware and resource requirements for training these transformer-based LMs from scratch, pre-trained transformer models will be used in my work, which is a common practice in the NLP community.

While many pre-trained models exist now and are available to use, one of the first pre-trained transformer models, bidirectional encoder representations from transformers (BERT) [Devlin et al., 2018] differentiates itself from the neural LMs that came before it by employing the transformer architecture [Vaswani et al., 2017]. When it was released, BERT achieved state-of-the-art performance on a wide range of NLP benchmarks. Examples of these benchmarks are GLUE [Wang et al., 2019] — a collection of nine NLP tasks, SQuAD [Rajpurkar et al., 2016] — a question-answering dataset, and SWAG

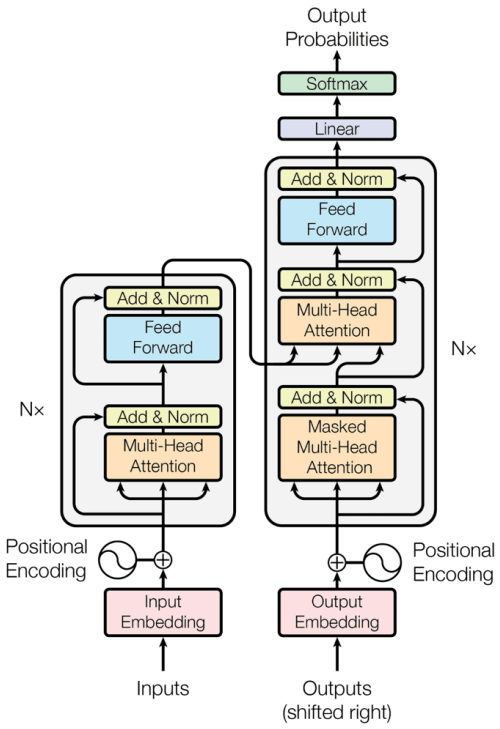


Figure 2.1: The transformer model architecture [Vaswani et al., 2017]

[Zellers et al., 2018]) — a commonsense reasoning benchmark. The LM was trained on two NLP tasks simultaneously; masked language modeling (MLM) and next sentence prediction (NSP).

MLM involves giving the LM a sentence with one of the tokens hidden — masked — and asking the LM to predict what the token is. An example of masked input to the model would be: "The [MASK] couldn't resist trying the lemonade." BERT would be asked to predict what token is being hidden by the [MASK] token. Training BERT on this task forces the model to leverage the bidirectional enhancement to capture the context on either side of the masked token before predicting it.

NSP focuses on predicting the next sentence in a pair of sentences. For example, "Julia went shopping. She bought a new pair of shoes." would be a correct sentence

pair, whereas "Julia went shopping. Bowling is a great way to pass the time." is an example of an incorrect sentence pair. This task lets BERT learn about the relationships between sentences.

Together, the bidirectional processing of text and the transformer architecture allow BERT to capture more nuanced contextual information. This allows the model to calculate an even more precise distribution of tokens when it comes to predicting the next token in a sequence. BERT also adds the the `[CLS]` and `[SEP]` special tokens to input sentences. The `[CLS]` token is added to the beginning of each sentence and provides an aggregated representation of the entire input sequence, making it useful for sequence-level tasks (which we'll see later!). The `[SEP]` token is used to differentiate between different segments or sentences within the same input. Throughout the rest of this paper, the `[CLS]` token is the only one of these special tokens that will reappear.

2.3 Embeddings

Word embeddings capture syntactic and semantic relationships between tokens, and are represented as n-dimensional numerical vectors. This vector representation allows ML models to learn relationships between tokens. For example, Mikolov et al. [2013] demonstrate this process with vector representations of *Paris*, *France*, and *Italy* in the following equation to determine — through analogical reasoning — an embedding that is very close to the true *Rome* embedding in vector space:

$$Paris - France + Italy = Rome$$

To us, the relationship is clear; Paris is the capital of France, and Rome is the capital of Italy. To ML models, the individual tokens "Paris" and "Rome", in a vacuum, may

not appear to have any relation at face value. By subtracting the *France* embedding from the *Paris* one, and by adding the *Italy* embedding, an embedding very similar to the *Rome* embedding can be learned, thus establishing that "Paris" and "Rome" appear in similar contexts, and that Paris is to France what Rome is to Italy (a capital city!).

There are two major different types of embeddings, sparse and dense. Sparse embeddings are high-dimensional representations with predominantly zero values, whereas dense embeddings are low-dimensional vectors that are able to capture complex relationships and similarities between tokens or phrases.

Sparse embeddings can be represented as the rows in a term-term matrix, shown in Table 2.5. For example, the sparse vector embedding for the token "the" (assuming the model is solely trained on the phrase "The llama couldn't resist trying the lemonade") is represented by the row in Table 2.5 corresponding to said token; $[0, 1, 0, 0, 1, 1]$. Note that the size of the matrix is the number of tokens in the vocabulary (in our example, 6), squared, as every token appears both on the x and y axis of Table 2.5. This relationship between the vocabulary size and the length of sparse embeddings contributes to both their high-dimensionality and the "sparse"-ness, with many values in each embedding being zero value.

Contrary to the vector of integers sparse embeddings provide, dense embeddings employ vectors where the vast majority of the embedding dimensions are non-zero value. Dense embeddings can not only capture syntactic and semantic relationships, but are also able to handle unseen tokens based on the context those tokens appear in. Commonly generated through neural language models, dense embeddings can be retrieved by training a neural language model on a large corpora, passing a single token to the LM, and retrieving the weights associated with the first hidden layer of the LM. These

	the	llama	couldn't	resist	trying	lemonade
the	0	1	0	0	1	1
llama	1	0	1	0	0	0
couldn't	0	1	0	1	0	0
resist	0	0	1	0	1	0
trying	1	0	0	1	0	0
lemonade	1	0	0	0	0	0

Table 2.5: A term-term matrix for the phrase: "The llama couldn't resist trying the lemonade"

weights are represented as a vector — a dense embedding.

As an example, if we want to get an embedding for a specific token from BERT, a pre-trained transformer model, we would pass the token to the LM and retrieve the weights of the entire last layer of the network after the token has been processed. BERT outputs a dense embedding that has 768 dimensions.

2.4 Generative Models

A generative text model can be as simple as passing textual input to a LM, asking the LM to return the top k next tokens to us, and randomly select one of those k tokens as the continuation of the input. This model sequentially "generates" the next token(s) to appear after an input phrase through random selection from a batch of k tokens. For demonstration purposes, "salt and" will be our input to this simple generative model

and Table 2.6 denotes the probabilities of certain tokens continuing the input.

Let's assume $k = 3$ for this example. If we take "pepper", "vinegar", and "water" (the top k tokens) from Table 2.6, the generative text model would have a 1/3 chance of selecting any of those tokens, despite the token "pepper" having the highest probability of coming next. We can't resort to only using the token with the highest probability because this would create an infinite loop of sorts, where the model's replies are predictable and can be manipulated by knowing which phrases to give the model in order to yield specific outputs. In this example, assuming the model selects *only* the continuation token with the highest probability, each time the model is prompted with "salt and" we *know* that it would continue the prompt with "pepper". By selecting one of the top k tokens, generative models can feel more dynamic and natural in its replies, avoiding stale and repetitive replies.

Generative Pre-trained Transformers (GPT) represent an advancement in the field of generative text models. GPT models are built upon the transformer architecture, which excels at capturing long-range dependencies and contextual information within text [Vaswani et al., 2017]. The transformer architecture's self-attention mechanisms enable GPT to understand the relationships between different tokens in a sentence, leading to human-like outputs that exhibit coherent and contextually relevant language generation [Vaswani et al., 2017, Radford et al., 2018]. While generative models excel at chatbot-esque interactions, they possess a wide array of versatile applications that extend well beyond chatbots. Their utility extends to machine translation [Dabre et al., 2020], text summarization [El-Kassas et al., 2021], and even story generation [Garbacea and Mei, 2020]. These models can aid writers, developers, and artists by offering suggestions, completing sentences, or even generating entire pieces of text. A

Token	**Probability**
pepper	0.20
vinegar	0.19
water	0.17
butter	0.15
ash	0.15
stone	0.04
sanctuary	0.03
food	0.03
seasoning	0.02
sugar	0.02

Table 2.6: The top k tokens (proposed by a language model) and their probabilities (computed by the model, rounded to two digits), sorted in descending order by probability. These tokens will be used by the generative model to continue the input "salt and".

group of researchers have even employed GPT models to simulate generative agents with believable human behaviour [Park et al., 2023]. The breadth of their applications demonstrates the potential for generative models to contribute significantly to a wide range of endeavours.

The GPT models from OpenAI[4] are more complex generative models that leverage the transformer architecture. The most recent models (GPT-3 [Brown et al., 2020] and GPT-4 [OpenAI, 2023]) from OpenAI have achieved near state-of-the-art results on numerous NLP tasks when compared to other models that have been fine-tuned on said tasks [Brown et al., 2020], with the ChatGPT[5] application taking the internet by storm. These models showcase remarkable capabilities in various language-related tasks, positioning them as leading players in the generative text model landscape [Brown et al., 2020, OpenAI, 2023]. These models are able to perform a multitude of NLP tasks very well, without requiring fine-tuning on each individual task.

LLMs that are capable of generating seemingly "human-written" text can be used for a wide variety of tasks and applications, some of which include chatbots and language translation. Whilst these models are powerful, they are not perfect and can often "hallucinate", or provide incorrect information that the model believes to be true. An example of this can be shown in a conversation I have with OpenAI's ChatGPT, shown in Figure 2.2. ChatGPT is taking my word that there was only one survivor of the sinking of the Titanic for granted here despite knowing that there were 712 survivors.

[4] `https://openai.com/`

[5] Found at `https://chat.openai.com/` at the time of writing.

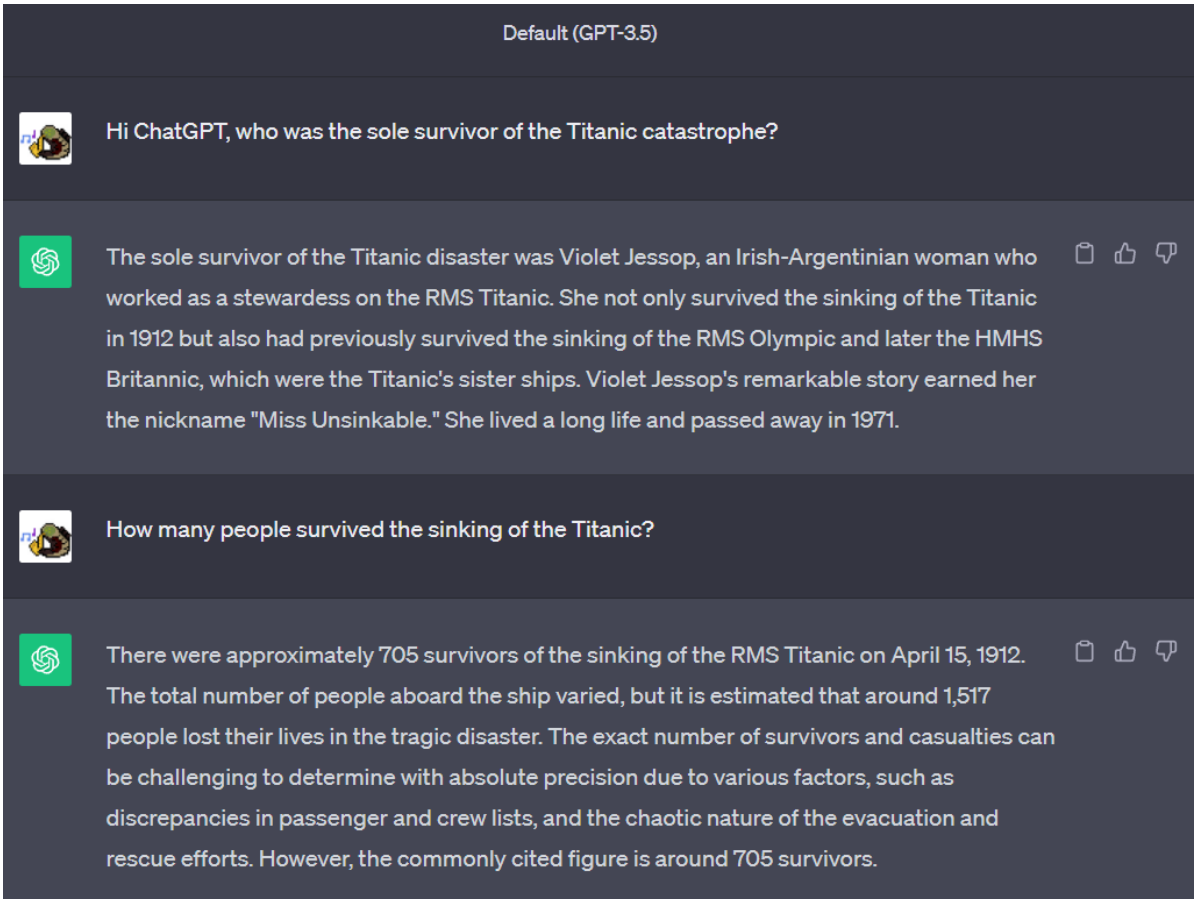


Figure 2.2: An example of ChatGPT, a large language model, hallucinating.

2.4.1 Applications of Generative Text

Generative text models can be applied to a variety of domains, not just to video game dialogue generation. This section covers three different domains generative text models have been applied to. This is by no means an exhaustive list, merely some examples to demonstrate the diversity of these models.

Video Games

As the motivation for this project stemmed from the desire to improve player immersion in video games using NLP, it's important to recognize games that have already been leveraging NLP. AI Dungeon is a "text-based adventure-story game you direct (and star in) while the AI brings it to life"[6]. Using generative text models, this game is able to create worlds, keep track of actions the player makes, and respond in unique ways.

Mental Health

Generative text models have been used to create psychotherapy chatbots [Das et al., 2022], presenting a non-traditional therapy method for individuals struggling with mental health. Wang et al. [2021] discovered that a generative text model fine-tuned on therapy session transcripts exhibited unique behaviour compared to a pre-trained, non fine-tuned, model. It also generated replies with lengths akin to that of therapist replies, when compared to the pre-trained model. Both the pre-trained model and fine-tuned model were likely to generate more negative and fewer positive outputs than the therapists [Wang et al., 2021].

In an attempt to generate schema-based stories that can be used to train classification

[6]Available at: `https://aidungeon.io/`

models, Lam et al. [2021] employed generative text models to assist with schema therapy — a psychological treatment technique. Schema therapy classifies patients to a schema mode — which represents their state of mind — based on the patient's thoughts and behaviour. With sub-optimal data, generative text models are still able to generate coherent stories with a minimum of 58.7% correctly classified samples, demonstrating the benefits these models can provide mental health professionals to ensure each patient gets the help they need.

Grammar Correction

With the understanding of natural language state-of-the-art LLMs possess, writing tools such as Grammarly[7] employ personalized generative LLMs to assist their users in writing. These personalized models can generate ideas, create job descriptions, and even propose rewrites in the same tone as the original text.

2.5 Prompt Engineering

Prompt engineering represents a departure from the conventional method of training models by providing explicit examples and labels in a traditional supervised learning setup. Rather than meticulously curating large datasets and engaging in extensive training processes, prompt engineering harnesses the power of existing, pre-trained models and adapts their behavior to suit specific tasks. The technique allows the emphasis to shift from providing extensive training data to embedding a clear description of the desired task within the input given to the LLM. This description outlines the objective, context, or instructions in a manner that guides the model's understanding of what's

[7] `https://www.grammarly.com/`

expected. Framing questions, providing context, or using keywords to guide the model's attention are merely a few examples of how prompt engineering can affect the initial query presented to a language model.

Once the task description is adequately explained and embedded into the input, the language model takes over and uses its learned knowledge and patterns to generate responses that align with the provided context. This process leverages the model's pre-existing knowledge, linguistic abilities, and understanding of relationships between tokens and phrases. The technique has been used across a variety of NLP tasks [Liu et al., 2023a], such as text classification [Hambardzumyan et al., 2021], mathematical reasoning [Reynolds and McDonell, 2021], and named entity recognition [Cui et al., 2021].

As prompting an LLM in different ways can drastically effect the quality of the output, I will be employing prompt-engineering techniques in my work. By tailoring a prompt to achieve the desired output, I can make the character the user converses with in my dialogue system feel unique in the manner in which they speak.

2.6 Long Term Memory

Part of the reason LLMs can't generalize to problems as well as humans is due to the input token limit. LLMs cannot process more text than their architecture allows for, but if that's the case, how can these models be expected to remember entire previous conversations or the background of a character they're emulating?

By providing an external database or cache [Wang et al., 2023] to an LLM, these models can work with much more text than their base un-augmented architecture allows for, essentially providing them with a long-term memory. With the amount of

information attaching an external database to an LLM provides, some incoming text may contradict text in the database, leading to the need for an automatic contradiction detection system.

2.7 Contradiction Detection and Natural Language Inference

The law of contradiction, stated by the Oxford Dictionary[8], is "a thing cannot be and not be at the same time," therefore two sentences can be deemed "contradictory" if they both cannot exist at the same time. The task of contradiction detection is derived from that of natural language inference (NLI), also known as recognizing textual entailment (RTE). For the purposes of this paper, we will exclusively use NLI to refer to the broader problem. NLI is a ternary classification problem that poses the question: Do two sentences contradict, entail, or remain neutral? For example, "most birds build nests in trees" and "the sun rises in the east" are two statements with a neutral relationship. These phrases cannot be contradictory as they do not discuss the same phenomena, and the fact that most birds live in trees doesn't imply anything about the sun rising in the east, and vice versa.

In contrast, contradiction detection is a binary classification problem, focusing solely on determining whether two sentences contradict each other or not. The phrases used in the NLI example — "most birds build nests in trees" and "the sun rises in the east" — are classified as `non-contradictory` as their relationship is neutral. To elaborate, all pairs of phrases with neutral or entailed relationships belong to the `non-contradictory`

[8] `https://doi.org/10.1093/OED/5575599456`

class, whereas two phrases that *do* contradict each other remain in the `contradictory` class.

In order to teach LMs how to distinguish between contradictory and non-contradictory pairs of phrases, contradiction specific token embeddings [Li et al., 2017] have been proposed. These embeddings aim to maximize the distance between embeddings of contradictory tokens. Current state-of-the-art embedding techniques leverage context to provide vector representations of tokens, but many contradictory tokens appear in similar contexts despite the true meaning of the sentence varying wildly. For example, this embedding technique places the representations of "always" and "consistently" close together, but both would be far away from the embedding representation of "never".

Other features are important when determining if two sentences contradict each other. Researchers Li et al. [2017] discuss three such features, but I will be highlighting the "number of negations" feature; the number of negation tokens across both sentences. This feature can provide insight into if two sentences contradict, as an odd number of negation tokens makes the probability of a contradiction being present increase. As an example, the phrases "I always shop on a Tuesday" and "I shop on Tuesday consistently" have 0 negation tokens and do *not* contradict each other, but the phrases "I shop on Tuesday consistently" and "I never shop on a Tuesday" have 1 negation token between them (never) and do contradict each other. This is not a hard and fast rule, but in a general sense it holds true [Li et al., 2017].

Liu et al. [2023b] discuss two different prompting techniques in order to teach an LLM to understand sarcasm; clash and question prompting. Assuming x is the input phrase, clash prompting uses unique masked prompts in an attempt to either agree with (in the case of a non-sarcastic input) or disagree with (in the case of a sarcastic input)

x. The LM attempts to predict the "[MASK]" at the end of each of said prompts[9]. Question prompting simply asks the LM directly if x is sarcastic or not by having the prompts formatted as yes-or-no questions[10]. Clash prompting asked open ended questions, making evaluating the quality of replies more difficult. To contrast, the benefit provided by question prompting is that the LM is only responsible for producing a binary "yes" or "no" output.

2.8 Persistent Homology

Persistent homology is a topological data analysis technique that identifies and quantifies topological features within data. There are several key dimensions persistent homology uses to provide a unique perspective on data. Only dimensions 0 and 1 will be employed in this research. Dimension 0 looks at connected components; clusters of data points that are connected to each other but not to other clusters. Dimension 1 captures loops, tunnels, and voids in the data. Loops are circular structures, paths connecting different components are tunnels, and voids are empty spaces enclosed by loops. An example of the construction of these features is visualized in Figure 2.3.

All features have a "birth" and "death" property, signalling when the feature was created and when it ceased to exist. As the radii (ϵ) increases around each data point, the edges eventually collide. When these edges collide, a feature is born. These features are visualized using a "persistence diagram" (also known as a "barcode"), of which an

[9]the five clash prompts are: "x Actually [MASK]", "x In reality it was [MASK]", "x, As a matter of fact, it was [MASK]", "x To tell you the truth, it was [MASK]", and "x In fact, it was [MASK]", where x is the input phrase, and the LM is trained to predict the correct label hidden by "[MASK]".

[10]The three question prompts are: "x Are you kidding? [MASK]", "x Are you sarcastic? [MASK]", and "x Are you ironic? [MASK]", where x is the input phrase, and the LM is trained to predict the correct label hidden by "[MASK]".

example can be seen in Figure 2.4. Wu et al. [2022] use these persistent homology (PH) vectors in their experiments to enhance the performance of their BERT-based classifier. I experiment with these features and analyze if they can have a positive impact on the natural language inference task I am tackling.

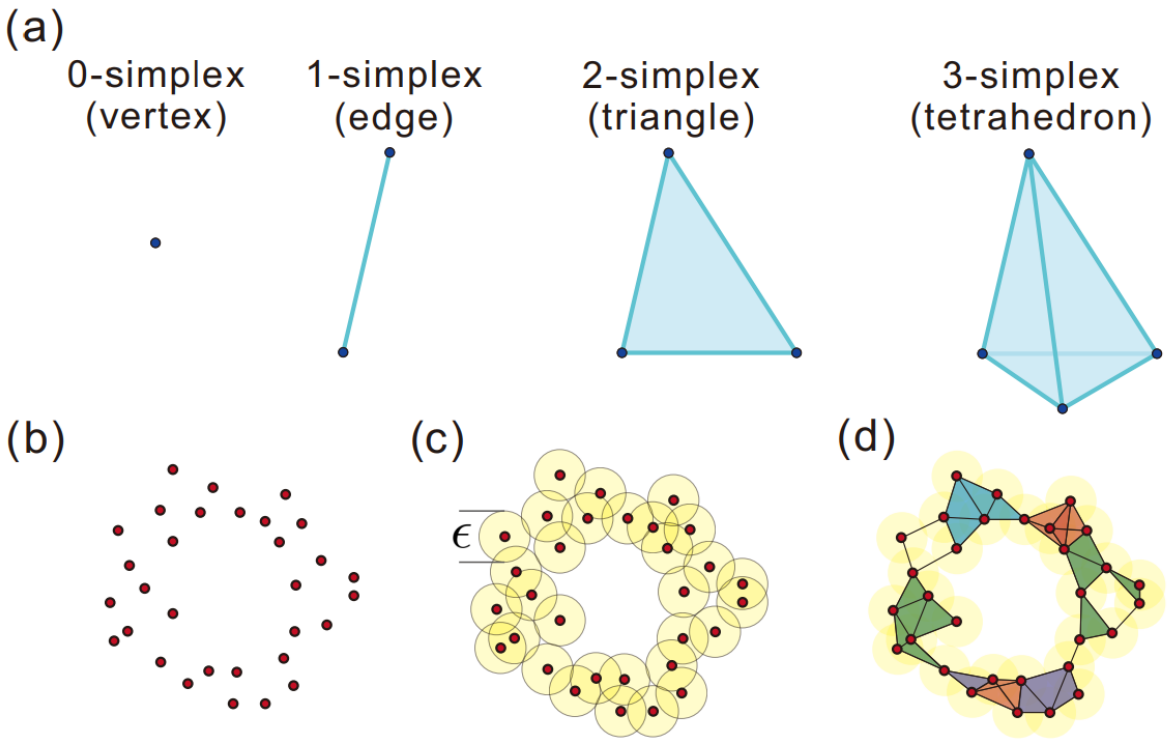


Figure 2.3: PH features and their construction [Huang et al., 2018]. (a) Construction of *k-simplices* — fully-connected sets of $k+1$ data points. (b) Scatterplot of example data points. (c) Using a distance metric to quantify the distance between points, ϵ, all data points within ϵ of each other receive an edge between them. (d) The final set of features is formed as a complex — set of simplices. The coloured regions represent different features within the complex.

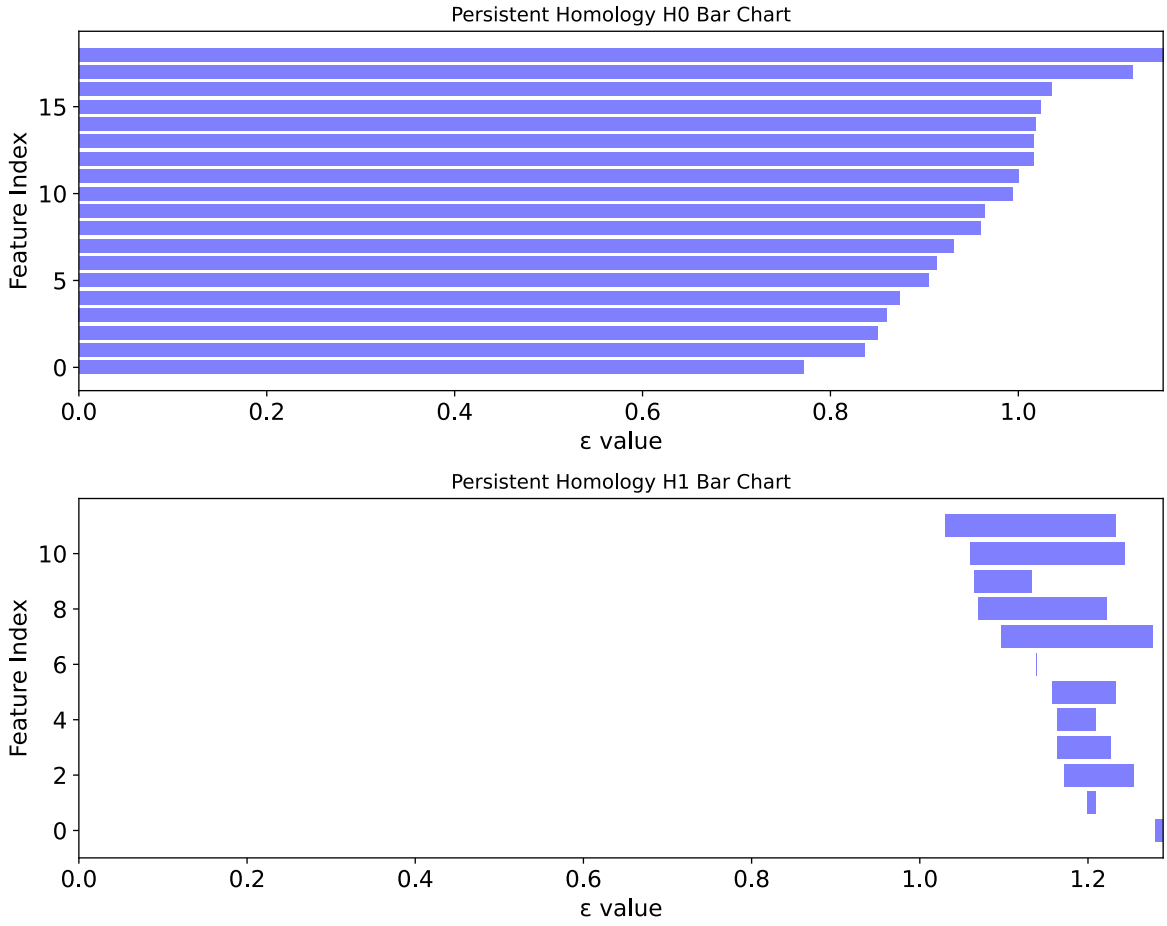


Figure 2.4: An example persistence diagram. The ϵ value on the $x - axis$ details the birth and death values of each feature. Dimension 0 features are displayed in the upper graph, whereas dimension 1 features are displayed in the lower one.

Chapter 3

The Wizard Behind the Project's Curtain: Data

3.1 Combating Memory Limitations

To combat the long-term memory issue presented by LLMs, ChatNPC, my dialogue system, employs the use of Pinecone[1], a vector database system for efficient storage and retrieval of embeddings. Using the cosine similarity score, the database returns the top k most similar embeddings based on the prompt the user provides the system. The user's initial prompt is also embedded and stored in the database. The retrieved embeddings, and the sentences they are associated with, are injected into the prompt to provide additional context to the LLM. The engineered prompt is passed to the LLM, with the response then being stored in the database and presented to the user.

[1] `https://www.pinecone.io/`

3.1.1 Characters

The characters that players can interact with through the system all have a biography defined for them in natural language. The history of each character and information about their hometown is loaded into the database through the process outlined in Section 3.1.2 and used in the context retrieval process outlined in Section 3.1.3.

3.1.2 Information Storage

Each record stored in the Pinecone database is made up of three components, an ID, metadata, and the embedding itself. The ID of each record indicates when it was added to the database (an ID value of 0 indicates the first record added). Each record's metadata has two components, the text that generated each embedding, and the type of entry to the database the text is. The text can be either a query from a user, an utterance from the character itself, or a sentence from the character's background information. The "type" of each record is either "query", "answer", or "background"; reflecting the source of the statement.

The embeddings are calculated using the fast and accurate Sentence-BERT (SBERT) embedding model, `all-MiniLM-L6-v2`. A sentence embedding model generates one embedding per input sentence as opposed to one embedding per input token. SBERT was implemented using the model found on the open-source platform HuggingFace[2]. The comprehensive list of datasets the model was trained on can be found on the model card (see the provided URL) but some include the WikiAnswers Duplicate question pairs [Fader et al., 2014], SearchQA [Dunn et al., 2017], and Natural Questions [Kwiatkowski et al., 2019] datasets.

[2] `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`

Fact Summarization

The text information stored in the knowledge base consists of full sentences that may each propose multiple facts.

Distilling the background information and dialogue utterances into the facts these phrases present aids in removing ambiguity from the knowledge base ("memory") of each character. With each record only storing a single fact, the amount of tokens retrieved from the knowledge base is reduced significantly, thus reducing the operating costs of the system. Having the knowledge base store facts and not whole sentences allows the system to better determine contradictory facts. As a concrete example of this process at work, the sentence

> *Caleb Brown is a friendly and hardworking baker who was born and raised in Quarlet.*

was split into the following facts (taken verbatim from the output of the fact rephrasing system):

- "Caleb Brown is friendly"
- "Caleb Brown is hardworking"
- "Caleb Brown is a baker"
- "Caleb Brown was born in Quarlet"
- "Caleb Brown was raised in Quarlet"

Poignancy

Researchers have previously leveraged a metric referred to as "poignancy" in other GPT-enhanced research [Park et al., 2023]. This metric asks the LLM to determine the "importance" of a given statement to a particular character. While Park et al. [2023] apply this technique in three unique ways by having the LLM determine the importance of an event, conversation, and thought I use the technique by asking the LLM to determine the significance of a particular fact to the proposed character. In order to ask the LLM how poignant a particular fact is to a character, it is presented with the following template:

> This is a brief description of $< 0 >$.
>
> $< 1 >$
>
> On a scale of 1 to 10, where 1 is purely mundane and 10 is extremely significant, rate the likely significance of the following information to $< 2 >$.
>
> Information: $< 3 >$
>
> Rate (return a number between 1 to 10): ONLY include ONE integer value on the scale of 1 to 10 as the output. Example output integer: 5

In the aforementioned prompt template, $< 0 >$ and $< 2 >$ are both replaced with a character's full name, $< 1 >$ is replaced with that same character's background information on one line, and $< 3 >$ is replaced with the fact in question. The example output and extra instruction to only include a single integer aid the model in only providing a single integer reply. In the interest of testing how the provided example biases the output of the LM, I tested this template with 5, 1, and 10 as the example scores. I'm happy to report this caused the LLM to output identical poignancy scores for each statement tested, displaying consistency in it's score assigning process.

The prompt, example output, extra instructions, and LLM parameters (such as limiting responses to only 1 token in length) result in the LLM outputting logical poignancy scores for different memories. For example; when asking the character Caleb Brown (a baker) if he knew of a cat stealing fish from the docks, the LLM gave the memory a score of 2 (fish aren't relevant to Caleb's profession). However, when presented with the fact: "Your wife is in the hospital" — a fact that is *significantly* more relevant to Caleb's life than a cat stealing fish — a poignancy score of 10 is assigned.

While that's all well and good, if Caleb's wife was in the hospital four years ago and has since recovered, Caleb probably shouldn't be expected to remember that traumatic experience each time he's asked about his wife. To try and provide these characters with some reprieve from their more haunting memories, let's incorporate a time data point into these memories.

Last Access Time

The characters ChatNPC allows you to converse with should have good — but not perfect — recall. Facts more recently presented to the character should be more prominent in memory — and more likely to be remembered — than older facts. To facilitate this behaviour, ChatNPC keeps track of the time at which a memory was last accessed by the memory retrieval system. The impact of the poignancy and the last access time information on the memory retrieval process is outlined in the Section 3.1.3.

The Structure of a Record

When records are created to be sent to Pinecone, they are initialized as Python dictionaries and are structured as shown in Figure 3.1 — where the ID of the record is the

total number of records in the knowledge base when a new one is added[3], and the value is the sentence embedding representation of the text present in the metadata. Each user's custom username is also saved to allow the filtering of agent memories based on the user the agent interacted with. Providing a username to each record provides two benefits for ChatNPC; conversations had with the same character but with different users are kept separate — not muddling memories — and only a single knowledge base is required for the system to operate efficiently — a distinct knowledge base for each user may significantly increase resource costs.

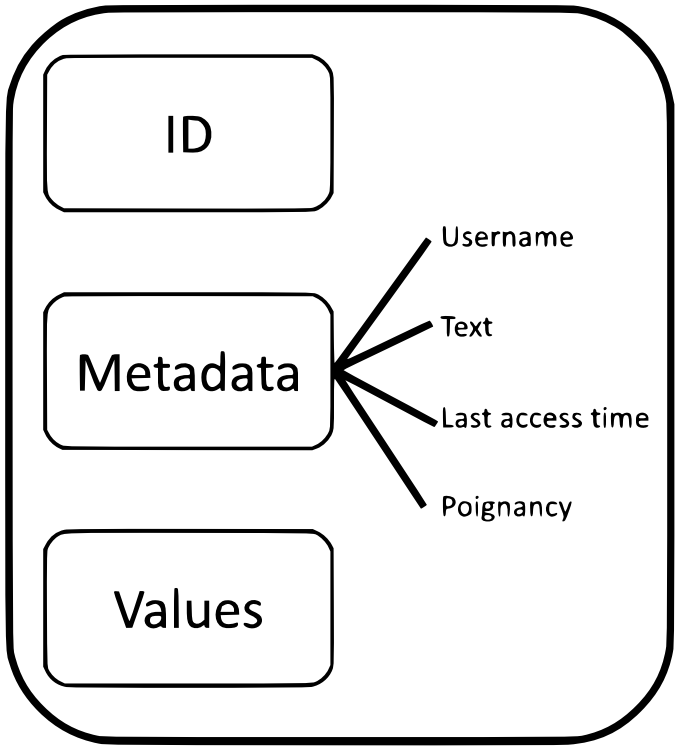


Figure 3.1: A visualization of a record as it is stored in the Pinecone database.

[3]When there are 0 records in the knowledge base, the first record to be added gets an ID of 0. The second record, as there is a single record in the knowledge base already, gets the ID 1, and so on and so forth.

3.1.3 Information Retrieval

The information retrieval mechanism in ChatNPC is designed to get the most impactful memories a character has to help them answer the user's query to the best of their ability. Two distinct steps occur before the final k — a hyperparameter of ChatNPC — memories are presented to the character the user is having a conversation with; querying the database and ranking the records. At a high level, the process involves retrieving all memories a specific character has, ranking all of said memories by leveraging three independent scores, and selecting the top k of these memories to aid in crafting the character in question's reply.

Querying Pinecone

With Pinecone storing all the memories each character possesses in a unique namespace[4] — a partition of the database that allows character information to be stored independently of one another — ChatNPC only requires the username of the user and name of the character the player is conversing with to access said character's stored memories.

Pinecone, leveraging the username and the character name, retrieves all records stored in the character's namespace — with the same username in said record's metadata. To kick-start the retrieval-from-Pinecone process, the user query, q, is converted from natural language to an embedding representation, q_e, using the sentence embedding model `all-MiniLM-L6-v2`[5] referenced in Section 3.1.2. When q_e is passed to the database, a score is calculated between q_e and all other embeddings, excluding those of type "query", stored in the database. The distance metric used is the cosine similarity, shown to work well when comparing embeddings, which is defined in Equation 3.1.

[4] `https://docs.pinecone.io/docs/namespaces`

[5] `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} \tag{3.1}$$

Equation 3.1: The cosine similarity equation, where A and B are vectors of the same length.

All records in the database, denoted D, are sorted in descending order by each record's associated embedding's cosine similarity to the query, and retrieved from the knowledge base. The retrieved records, D_r keep track of their similarity score during this process, allowing the score to be leveraged later. The text element of the metadata of each of the D_r records is extracted for use in prompt engineering (elaborated upon in Section 6.6).

Type "query" records are excluded because when a character is asked the same question multiple times, D_r becomes over-saturated with records that consist of near-identical questions. These records do not provide relevant information that the character requires in order to answer the question being asked. For example, assuming the questions and answers are saved, if a player has asked the same character "Where's the best bread in town?" at least $k + 1$ times — where k is the number of records ChatNPC uses to provide the character context to adequately respond to the user's query — eventually the retrieval process would only present records consisting of "Where's the best bread in town?" and not an actual answer to the question.

Ranking Records

The second step in finding a character's k most impactful memories is ranking them. Whilst Pinecone ranks the records by the cosine similarity score, referred to as "rel-

evance" from here on out, that alone is an insufficient metric to determine the most impactful memories. Two other pieces of data that are saved with each memory — the poignancy and last access time data — which I can use alongside the relevance metric to create a three-dimensional score for each record [Park et al., 2023].

Whilst the relevance and poignancy metrics — integers — can easily be incorporated into a calculation, how is the last accessed time data — saved in the "YY-mm-dd HH:MM:SS" format — used in the record ranking process? In Section 3.1.2, it is noted that NPCs should have good but not perfect memory. This behaviour is achieved through an exponential decay function. This function analyzes when the memory was most recently accessed and the time at which ChatNPC is searching for memories to provide context for the user query. With that information in hand, ChatNPC calculates the difference between the two times (Δ), and calculates an exponential decay score — henceforth referred to as "recency" — using a decay rate of 0.01 (γ). This calculation is outlined in Equation 3.2.

$$\text{recency score} = e^{-\gamma \cdot \Delta} \tag{3.2}$$

The product of the relevance, poignancy, and recency scores yields a single metric — the impact score — that can be used to rank the impact of each memory in the namespace of a particular character. Once the impact score is calculated for each record, they are sorted in descending order by impact score, and the top k records, i_k, are used to provide context for the character's reply to the user's query.

3.2 Datasets

Three publicly available datasets are employed to test classification models on the natural language inference (NLI) NLP task. Agents that users have conversations with through a dialogue system have access to certain information and facts. While agents believe the facts they have access to are the truth, the facts themselves could be inaccurate, leading to agents opposing the information being presented to them by the user. For example, if agent one (A_1) believes that the favourite food of agent two (A_2) is hot dogs, but the user tells A_1 that A_2's favourite food is actually stir-fry, how does the agent know which "fact" is true? An automatic contradiction detection system can parse the context being provided to A_1 and determine if there are any contradictions between A_1's knowledge and the information being presented to it by the player. If a contradiction is detected, agent A_1 asks the user to choose which fact is true, allowing it to update its knowledge base accordingly.

I will be breaking down the different datasets used to train and evaluate potential contradiction detection models. The models themselves and their performances will be discussed in Chapter 4. The best performing model will be used in the dialogue system to compare potentially contradictory pairs of facts.

3.2.1 Sentences Involving Compositional Knowledge

I employ the Sentences Involving Compositional Knowledge (SICK) dataset used in Task 1 of SemEval 2014, "Evaluation of compositional distributional semantic models on full sentences through semantic relatedness and textual entailment" [Marelli et al., 2014] when working on NLI. The data, retrieved through HuggingFace[6], is in English

[6] `https://huggingface.co/datasets/sem_eval_2014_task_1`

Label	Sentence Pair
Entailment	*A:* Two teams are competing in a football match
	B: Two groups of people are playing football
Contradiction	*A:* The brown horse is near a red barrel at the rodeo
	B: The brown horse is far from a red barrel at the rodeo
Neutral	*A:* A man in a black jacket is doing tricks on a motorbike
	B: A person is riding the bicycle on one wheel

Table 3.1: Examples of sentence pairs from the SICK dataset with their gold labels.

and contains pairs of sentences from both image annotation and video descriptions. Training, validation, and testing splits were provided with the download of the dataset. The number of records in each split is outlined in Table 3.4. The dataset includes both a "relatedness score" and an inference class, which contains the *neutral*, *entailment*, or *contradiction* label for each record. Only the inference class will be employed as I am using the dataset to work on the NLI problem. An example of a pair of sentence and their gold label can be seen in Table 3.1. This dataset will be referred to as the "SICK" dataset for the remainder of this paper.

3.2.2 Stanford Natural Language Inference

The Stanford Natural Language Inference (SNLI) dataset [Bowman et al., 2015][7] is a well-established English NLI dataset which will be employed to evaluate the performance of my presented models. Records consist of human-written sentence pairs that were manually annotated. Some records were assigned a label of "-", indicating that annotators did not reach a consensus on the value of the inference label for a particular record. These records were removed from the training, validation, and test sets, along with certain records that caused encoding issues due to characters that were unable to be encoded in `UTF-8`. Due to hardware limitations and the expensive nature of training, I am only working with a subset approximately 10% the size of the original SNLI training set, putting my model at a disadvantage when comparing to the SNLI baseline, which was trained on the entirety of the SNLI training data. The original SNLI training set, validation set, and test sets had $550,152$, $10,000$, and $10,000$ records, respectively. My models are evaluated on the unaltered test sets, identical to the SNLI baseline model I compare to. Examples of some records in the dataset can be seen in Table 3.2. Table 3.4 breaks down the total number of records remaining in the training, validation, and testing SNLI datasets post-cleaning.

3.2.3 Multi-Genre Natural Language Inference

The Multi-Genre Natural Language Inference (MultiNLI) dataset [Williams et al., 2018][8] is an Egnlish dataset modeled on SNLI, including more sentence pairs and introducing different "genres" (phone speech, letters, fiction, etc.), which indicates where pairs

[7] `https://nlp.stanford.edu/projects/snli/`

[8] `https://cims.nyu.edu/~sbowman/multinli/`

Label	**Sentence Pair**
Entailment	*A:* A soccer game with multiple men playing
	B: Some men are playing a sport
Contradiction	*A:* A black race car starts up in front of a crowd of people
	B: A man is driving down a lonely road
Neutral	*A:* A smiling costumed woman is holding an umbrella
	B: A happy woman in a fairy costume holds an umbrella

Table 3.2: Examples of sentence pairs from the SNLI dataset with their gold labels.

originate from. With the original dataset being comprised of approximately 433,000 crowd-sourced and annotated sentence pairs with textual entailment labels, MultiNLI has also been used to train sentence embedding models [Reimers and Gurevych, 2019] and is one of many datasets (including SNLI) to tune instruction-based models[9] such as FLAN [Wei et al., 2022]. Similar to the SNLI training set, I only use a subset of the MultiNLI training set when training my models. I still compare to the MultiNLI baseline model which was trained on the entirety of the training data.

The original MultiNLI dataset has training, matched validation (M.Valid), mismatched validation (Mm.Valid), matched test (M.Test), and mismatched test (Mm.Test) datasets. The MultiNLI training set has 392,702 records, whereas M.Valid, Mm.Valid, M.Test, and Mm.Test all have 10,000 records each. The matched validation and test sets use the same genres of text as the training data, whereas the mismatched development and test set use genres that do *not* appear in the training data. As with the SNLI dataset, I train my models on only a subset of the original MultiNLI training set. I removed some records that caused encoding issues (characters with accents, certain currency symbols, etc.). The test sets are not publicly available and can only be accessed through a Kaggle competition[10][11], requiring us to submit my model predictions to each competition to receive a score on each test set. The dataset features records with a fourth class label, denoted by "-", indicating annotator disagreement on that specific entry. For my purposes, these records are removed from the training and validation data.

[9]These models are specifically designed to process and act upon instructions or commands provided in text or another format.

[10]M.Test Set Competition: `https://www.kaggle.com/c/multinli-matched-open-evaluation`

[11]Mm.Test Set Competition: `https://www.kaggle.com/c/multinli-mismatched-open-evaluation`

Table 3.4 breaks down the final count of the number of records in each training, validation, and test set used in my experiments. Examples of records in the dataset can be seen in Table 3.3.

Label	**Sentence Pair**
Entailment	*A:* uh-huh how about any matching programs *B:* What about matching programs?
Contradiction	*A:* Your speeches are inflammatory. *B:* Your speeches are so calming.
Neutral	*A:* The Old One always comforted Ca'daan, except today. *B:* Ca'daan knew the Old One very well.

Table 3.3: Examples of sentence pairs from the MultiNLI dataset with their gold labels.

	SICK			**SNLI**			**MultiNLI**		
	Train	*Valid*	*Test*	*Train*	*Valid*	*Test*	*Train*	*M. Valid*	*Mm. Valid*
Neutral	2,536	282	2,793	18,276	3,235	3,219	3,719	3,093	3,129
Entailment	1,299	144	1,414	18,341	3,329	3,368	4,008	3,438	3,463
Contradiction	665	74	720	18,318	3,278	3,237	4,166	3,180	3,240
Total	4,500	500	4,927	54,935	9,842	9,824	11,893	9,711	9,832

Table 3.4: This table displays the class splits between the different datasets, post-cleaning, used to train and validate my models.

Chapter 4

Unmasking Jekyll and Hyde: The Art of Contradiction Detection

4.1 Context Pool Saturation

When the D_k records[1] are extracted they are added to what I've dubbed the "context pool", which is used by the LLM at the core of the system as context to answer a user's question. Preliminary testing of the dialogue system brought an issue to light — despite ignoring the user queries when searching for D_k (discussed in Section 3.1.3), the context pool would still become over-saturated with similar sentences. Examples of similar sentences would be denying knowing the answer to a question ("I do not know where the best bread in town is") or similar sentences that discuss an alternate topic ("The best muffins in town are found at the bakery"). If a character denies knowing the answer, replies with a similar answer, or a combination of both, $k + 1$ times, even after they are told the correct answer, the cosine similarity isn't a sufficient technique

[1]The records extracted from Pinecone by the retrieval process.

to filter out these replies. This discovery made me look into automatic contradiction detection. A proposed model should be able to determine if the context provided and the information in the query have either a contradictory or non-contradictory relationship.

4.2 Contradiction Detection Models

A variety of experiments are performed to determine the contradiction detection model that best fits into my dialogue system. The models will be evaluated on their performance when subject to the natural language inference (NLI) task.

4.2.1 Natural Language Inference

Natural language inference (NLI) is a task that assesses relationships between sentence pairs to be either neutral, entailed, or contradictory. This problem involves analyzing the semantic relationships between two sentences and leverages the nuance of language to assign a label to the pair.

In my experiments, four unique LLMs were employed as the foundation for my vanilla (non-augmented) models: `bert-base-uncased` [Devlin et al., 2019], `roberta-base` [Liu et al., 2019], `roberta-large` [Liu et al., 2019], and `all-MiniLM-L6-v2` (SBERT) [2]. The code for all experiments is available at `https://github.com/VeiledTee/ChatNPC`. `roberta-base`[3] and `roberta-large`[4] are open-source BERT-based models that have been trained on more data and solely on the MLM task. SBERT is a BERT-based sentence embedding model, which provides a single embedding for an entire sentence, instead of one label for each token in the input phrase. The data were presented in pairs

[2] `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`
[3] `https://huggingface.co/roberta-base`
[4] `https://huggingface.co/roberta-large`

of sentences, and each of these LLM-based models generated two embeddings, one for each sentence, for subsequent classification. All models were designed as ternary-class models, predicting either the *neutral*, *entailment*, or *contradiction* class for each pair of sentences they encountered.

Each model was exclusively trained, validated, and tested on a single dataset at a time. For instance, models trained on the SICK dataset underwent validation and testing solely on the SICK validation and test sets. It's important to note that all vanilla models were assessed separately for each dataset, resulting in distinct experiments. Augmented models trained on the SNLI or MultiNLI datasets were evaluated exclusively on the same dataset's validation and test sets.

4.2.2 Architecture

This section details the varying architecture of the contradiction detection models. The following sections will detail the architecture of all models that employ the embedding model reflected by their names.

4.2.3 Vanilla Model Architecture

Each unaugmented (vanilla) model takes a pair of sentence embeddings as input for classification purposes. For the evaluation of my vanilla models, each will be subjected to the SICK, SNLI, matched test (M.Test), and mismatched test (Mm.Test) test sets to establish a score for each model. These results, and the baselines established in the work inspiring my augmentations, will be compared to those achieved by my augmented models.

BBU is a BERT classifier that employs `bert-base-uncased` as the LLM, *RoBERTaB*

leverages `roberta-base`, and *RoBERTaL* uses `roberta-large`. Four other models were also evaluated, a support-vector machine (SVM) using `bert-base-uncased` generated embeddings (*SVM-BBU*), another SVM using `all-MiniLM-L6-v2` [Reimers and Gurevych, 2019][5] generated embeddings (*SVM-SBERT*), and an *XGBoost* random forest model that works with `bert-base-uncased` embeddings. When discussing the hyperparameter configuration that led to each model's best performance, all parameters not directly mentioned are assigned their default value.

BERT Classifiers

The BERT classifiers embed each sentence in the pair of input sentences and use said embeddings to predict which class a pair of sentences belong to. These classifiers are implemented using `AutoModelForSequenceClassification` and `AutoTokenizer` from the `transformers` [Wolf et al., 2020] Python library. The models take lists of sentences, generates the embeddings and an attention mask for each sentence, and uses that information to predict if a pair of sentences are one of the three classes available in an NLI task. Each model's learning rate is set to 2×10^{-5}, max sequence length is set to 128, batch size is set to 32, and each model is trained for 10 epochs. Due to the ternary class nature of the problem, the number of classes in the initialization of the model is 3.

Support Vector Machines

The SVM models, despite leveraging different embedding models, have identical architectures. Using the embeddings of the input sentences as data points, the model attempts to maximize the space between the data points associated with the three classes present in

[5] `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`

NLI tasks. The SVM models were implemented using the `SVC` class from `scikit-learn` [Pedregosa et al., 2011]. After tuning the hyperparameters on the validation data, the *SVM-BBU* model performed best with a linear kernel and a C value[6] of 0.1 across all test sets, and the *SVM-SBERT* performed best with a linear kernel and a C value of 1.0 across all test sets.

XGBoost

The *XGBoost* [Chen and Guestrin, 2016] model sequentially creates decision trees, allowing each tree to correct the output of the previous one. The classifier also utilizes weights assigned to each independent input variable, and adjusts the value of the weights assigned to incorrectly classified variables. The model was trained on the `[CLS]` token embedding yielded by `bert-base-uncased` [Devlin et al., 2019] for each sentence in the input pair. Once again, using the validation set for hyperparameter tuning, the *XGBoost* model performed the best with a learning rate[7] of 0.2, a max depth[8] of 4, and 500 estimators[9].

4.2.4 Augmented Model Architecture

The different features employed to augment my vanilla models are discussed in this section. By passing the number of negations feature, persistent homology (PH) vectors, or both, to the LLM at the core of each model, I attempt to improve model performance on NLI tasks.

[6] Controls the trade-off between a smooth decision boundary and the correct classification of training points — larger values may cause overfitting and smaller may cause underfitting.

[7] Controls the size of the steps the model takes to minimize the loss function

[8] Controls the complexity of the model.

[9] Each of the 500 trees are built sequentially to correct the errors of the previous ones.

A single integer represents the number of negations across both input sentences. Two PH vectors will be generated using the `Ripser` [Bauer, 2021] library for each input sentence, a dimension 0 vector of shape $[260, 3]$ and dimension 1 vector of shape $[50, 3]$. In total, four PH vectors will be generated for each pair of input sentences.

It should be noted that the architecture of each model is not altered when testing on different test sets; the same architecture is retrained from scratch on the training data for each dataset prior to testing.

Number of Negation Words

BBU_Neg is the unaltered `bert-base-uncased` model with the "number of negations" feature passed to the classifier alongside the embedding representation of both input sentences, from which the predictions are extracted, a softmax is performed, and the final labels are presented. The *RoBERTaB_Neg* model architecture is identical to that of *BBU_Neg*, except BERT is swapped for RoBERTa. All negation feature augmented models were tested on all four test sets.

Persistent Homology Vectors

Two classifiers were employed to supplement the PH models; a multi-layer perceptron (MLP) and an SVM. The MLP is a three-layer classifier that takes a $3,692$ ($3,693$ in the case of the *Neg_PH* models to account for the negation feature) element vector as input, and passes it through three 300 element layers, with a dropout[10] of 0.1 in the final layer, and a three-element output vector. The SVM uses a linear kernel and $C = 1.0$ for both LLMs. Through more hyperparameter tuning, the SVM classifier performs best

[10]Helps to prevent overfitting and improve generalization.

on the SICK validation set with a linear kernel and when $C = 0.1$. The SVM head uses embeddings generated by the LLM at the core of the model it is attached to, either `bert-base-uncased` or `roberta-base`.

The PH features are generated by passing the `[CLS]` token embeddings for each sentence in the input pair to the `Ripser` library. In previous PH-natural language work, [Wu et al., 2022] determine the optimal amount of dimension 0 and dimension 1 vectors is 260 and 50 respectively. Leveraging this insight, my models calculate the 260 dimension 0 features and 50 dimension 1 features generated for each `[CLS]` token per sentence (two features per sentence, four features per input sentence pair). These vectors are comprised of three values, the "birth" of the feature, the feature's "death" time, and the "persistence" value of the feature ($birth - death$). Sorted in descending order by persistence value, the most long-standing vectors are found at the top of the pile for each dimension. The top 260 are lifted from the dimension 0 vectors and the top 50 are skimmed off the dimension 1 pile. After getting the top vectors for *both* sentences in the pair, they are all passed to the classifier model in conjunction with the same embeddings used to generate said PH vectors. The classifier then outputs a three-element array which is subject to a softmax to determine which class the pair belongs to. The SVM classifier takes the same input as the MLP one, but skips the softmax step and outputs the class the model believes the record belongs to. My PH models will only be evaluated on the SICK test set as hardware limitations have prevented the calculation of PH vectors for all of the records in the SNLI and MultiNLI datasets.

Combined Feature Models

Dubbed "combined" models, my *Neg_PH* models combine both augmentations previously discussed in this Section. The PH vectors are calculated in the same manner, by taking the embedding representation of the `[CLS]` tokens from each sentence, passing them to the `Ripser` library, and skimming the top 260 dimension 0 features and top 50 dimension 1 features off the top. The number of negation words is also identical to the feature described in Section 4.2.4. The MLP and SVM classifiers are used in these models too, with identical architecture and hyperparameters as described in Section 4.2.4. The number of negation words feature is appended to the very end of the final PH vector before the data is passed to the classifier. As with my PH models, my combined models will only be evaluated on the SICK test.

4.2.5 Enhancements

Borrowing from the computer vision field of computer science, a one-dimension convolution layer was added to the beginning of a model in an attempt to capture both global and local features of token embeddings. A feature incorporating the total number of negation words across both input sentences, the `num negation` feature, a single integer representing the number of negation words (no, not, don't, etc.) across *both* input sentences, was also included. Li et al. [2017] employed a model trained and evaluated on the SICK dataset that incorporates both of the previously mentioned enhancements. The researchers' model outperformed previous accuracy benchmarks on the dataset by 6.1%.

Negation Word Count

A single integer represents the number of negations across both input sentences. *BBU_Neg* is the base `bert-base-uncased` classifier with the "number of negations" feature passed to the classifier alongside the embedding representation of both input sentences, from which the predictions are extracted, a softmax is performed, and the final labels are presented. The *RoBERTaB_Neg* model architecture is identical to that of *BBU_Neg*, except BERT is swapped for RoBERTa. All negation feature augmented models were tested on all four test sets.

Persistent Homology Vectors

The PH features are generated by passing the `[CLS]` token embeddings for each sentence in the input pair to the `Ripser` library. In previous PH-natural language work, Wu et al. [2022] determined the optimal amount of dimension 0 and dimension 1 vectors is 260 and 50 respectively. Leveraging this insight, my models calculate the 260 dimension 0 features and 50 dimension 1 features generated for each `[CLS]` token per sentence (two features per sentence, four features per input sentence pair). These vectors are comprised of three values, the "birth" of the feature, the feature's "death" time, and the "persistence" value of the feature ($birth - death$).

The top 260 are lifted from the dimension 0 vectors and the top 50 are skimmed off the dimension 1 pile. After getting the top vectors for *both* sentences in the pair, they are all passed to the classifier model in conjunction with the same embeddings used to generate said PH vectors. The classifier then outputs a three-element array which is subject to a softmax to determine which class the pair belongs to. The SVM classifier takes the same input as the MLP one, but skips the softmax step and outputs the class

the model believes the record belongs to. My PH models will only be evaluated on the SICK test set as hardware limitations have prevented the calculation of PH vectors for all of the records in the SNLI and MultiNLI datasets.

4.3 Results

In this section I discuss the results achieved by my vanilla models and their augmented versions, and compare them to my vanilla models performance and the established baselines from other researchers.

4.3.1 Vanilla Models

The baselines for each vanilla (out-of-the-box) model, by dataset, are presented in Tables 4.1, 4.2, and 4.3. While the accuracy, F1-score, precision, and recall are provided for my models for both SICK and SNLI test set results, both M.Test and Mm.Test from the MultiNLI dataset only provide accuracy as the scoring metric. As shown in Tables 4.1, 4.2, and 4.3, the three vanilla BERT models outperform all other vanilla models and outperform the baselines provided by [Li et al., 2017] on the SICK test set and by [Wang et al., 2021] on the SNLI test set.

While *RoBERTaL* outperformed *RoBERTaB* by 3.66% and 2.85% on the MultiNLI M.Test and Mm.Test test sets respectively (Table 4.1), it required more than ten times the training time. Combined with my hardware limitations, the minor difference between the *RoBERTaB* and *RoBERTaL* model performances on the SICK test set (0.45%), and the fact that *RoBERTaB* outperformed *RoBERTaL* on the SNLI test set by 0.54% leads me to leverage the `roberta-base` LLM instead of `roberta-large` as the LLM

foundation of the augmented *RoBERTa* models. `bert-base-uncased` will be used as the LLM foundation my augmented *BBU* models rely on.

The contradiction specific token embedding model presented by [Li et al., 2017] achieved an accuracy of 84.27% on the SICK dataset. This result was surpassed by my vanilla models *BBU*, *RoBERTaB*, and *RoBERTaL*. The best performing model on the SNLI dataset, according to the project's website[11], at the time of writing is the EFL model [Wang et al., 2021], which achieved an accuracy of 93.1%, which my vanilla *RoBERTaB* model narrowly surpassed by 1.17%. [Wu et al., 2022] evaluate their PH vector augmented models on the MultiNLI dataset, recording accuracy scores of 85.4% (+0.03% compared to their non-PH-ified BERT model) on the M.Test set and 85.9% (identical to the non-PH BERT) on the Mm.Test set. None of my vanilla models were able to surpass this baseline score.

4.3.2 Augmented Models

In the section I discuss the performance of my augmented models outlined in Section 4.2.4.

The Negation Feature

Incorporating the negation feature into my classifiers, specifically the *RoBERTa* and *BBU* models, yielded improvements of 0.33% (Table 4.4) in comparison to the *RoBERTaB* model's performance, and a maximum gain of merely 0.18% over the baseline *BBU*

[11] `https://nlp.stanford.edu/projects/snli/`

Model	**Matched**	**Mismatched**
Wu et al. [2022]	**85.4%**	**85.9%**
BBU	72.68%	70.06%
RoBERTaB	80.36%	80.22%
RoBERTaL	84.02%	83.07%
SVM-BBU	50.62%	50.07%
SVM-SBERT	44.01%	45.15%
XGBoost	30.90%	30.20%

Table 4.1: My scores for each model assessed in the experiments on the MultiNLI dataset. All scores are the results achieved by my models on both the M.Test and Mm.Test sets. The best results in each column are bolded. The BERT+TDA model from [Wu et al., 2022] is the baseline I'm comparing to.

Model	**Accuracy**	**F1-Score**	**Precision**	**Recall**
[Wang et al., 2021]	93.1%	N/A	N/A	N/A
BBU	92.36%	0.9263	0.9263	0.9264
RoBERTaB	**94.27%**	**0.9427**	**0.9432**	**0.9427**
RoBERTaL	93.73%	0.9376	0.9381	0.9373
SVM-BBU	62.46%	0.6184	0.6127	0.6246
SVM-SBERT	55.64%	0.5508	0.5461	0.5564
XGBoost	76.93%	0.7548	0.7619	0.7693

Table 4.2: Baseline scores for each LLM model assessed in the experiments on the SNLI dataset. All scores are the average of 30 iterations. The best results in each column are shown in bold. The baseline I'm comparing to is that established by the Entailment as Few-shot Learner (EFL) technique [Wang et al., 2021].

Model	**Accuracy**	**F1-Score**	**Precision**	**Recall**
[Li et al., 2017]	84.27%	N/A	N/A	N/A
BBU	85.24%	0.8529	0.8560	0.8524
RoBERTaB	88.90%	0.8892	0.8908	0.8890
RoBERTaL	**89.35%**	**0.8925**	**0.8935**	**0.8915**
SVM-BBU	79.62%	0.7957	0.7970	0.7962
SVM-SBERT	73.68%	0.7283	0.7347	0.7368
XGBoost	63.89%	0.6304	0.6318	0.6389

Table 4.3: Scores for each LLM model assessed in the experiments on the SICK dataset. All scores are the average of 30 iterations. The best results in each column are shown in bold. The baseline I'm comparing to is the CNN:CWE* + shallow features model presented by [Li et al., 2017].

model. Nonetheless, it is noteworthy that this feature had a detrimental impact on the accuracy predictions of models using the `bert-base-uncased` language model as their foundation, with reductions of up to 24.43%.

The number of negations feature improved the vanilla RoBERTa models' performance on the SICK, SNLI, M.Test, and Mm.Test test sets by 0.11% (Table 4.5), 0.04% (Table 4.6), 0.33% (Table 4.4), and 0.02% (Table 4.4) respectively. While *RoBERTaB_Neg* was the best-performing of my models on the MultiNLI test sets, it unfortunately fell 4.71% short to [Wu et al., 2022] on the M.Test set, and 5.66% short on the Mm.Test set.

Incorporating the number of negations feature into my vanilla *BBU* model caused the *BBU_Neg* model to only outperform the vanilla BBU model on the SICK dataset by 0.18% (Table 4.5). The negation-augmented model performed poorer than it's vanilla counterpart on the SNLI, M.Test, and Mm.Test test sets, with the accuracy dropping 0.27% (Table 4.6), 11.71% (Table 4.4), and a whopping 24.43% (Table 4.4) respectively. Upon analysing the predictions made by *BBU_Neg* after submission to the MultiNLI Kaggle competition, the model picked the *entailment* class for every record. The performance changes previously outlined may be due to the classifier interpreting this feature as noise. Appending the negation count integer to the two 768-dimensional embeddings of the `[CLS]` tokens in each sentence did not appear to substantially influence the model's decision. As displayed by the SNLI and both MultiNLI test set results, this feature can negatively impact a model's performance.

Persistent Homology Vectors

When first experimenting with my PH-augmented models on the SICK dataset, I employed MLP classifiers. Unfortunately, when compared to my vanilla models, the results

were underwhelming (see Tables 4.4, 4.5, and 4.6), likely due to the limited data available in the SICK dataset. In an attempt to improve model performance, the MLP classifier was swapped out for an SVM. An SVM was chosen as both vanilla models outperformed my vanilla XGBoost model on three of the four test sets the vanilla models were evaluated on.

Table 4.5 presents the performance of four PH-based models, both with MLP and SVM classifiers. Notably, the SVM-augmented variations outperformed the MLP models, over doubling their accuracy. Despite this improvement, my PH-based models fall short of my vanilla model and my negation-feature model results.

These observations raise questions about the amount of training data available for my MLP classifiers, as the SICK dataset contains only $4,500$ training records. They also highlight the performance disparities between MLP and SVM classifiers in my PH-based models. Before my investigation into the usefulness of PH vectors can conclude, it's essential to pair them with the number of negations feature and attempt to determine if these vectors are merely interpreted as noise, or if they can be leveraged to draw meaningful insights from text data.

Combined Models

The combined models; *BBU_Neg_PH_MLP*, *BBU_Neg_PH_SVM*, *RoBERTaB_Neg_PH_MLP*, and *RoBERTaB_Neg_PH_SVM* combine the number of negation and PH features into a single classifier. The architecture, outlined in Section 4.2.4, enables my combined models to harness both augmentations outlined in this work.

Table 4.5 presents the results of each LLM and classifier combination on the SICK dataset. Notably, among the combined models, the MLP classifier variant on both LLMs

performed identically to the *RoBERTaB_PH_MLP* model. This observation implies that the models might view the PH vectors and negation feature as noisy, possibly conflicting with the information conveyed by the embeddings, thereby yielding similar results.

While the *BBU_Neg_PH_MLP* model shows a 14.07% improvement compared to *BBU_PH_MLP*, the other combined models (*BBU_Neg_PH_SVM*, *RoBERTaB_Neg_PH_MLP*, and *RoBERTaB_Neg_PH_SVM*) do not fare as well, performing worse than their PH-augmented counterparts. Additionally, the identical performance of *BBU_Neg_PH_MLP* and *RoBERTaB_Neg_PH_MLP* compared to *RoBERTaB_PH_MLP* suggests that the negation feature may not only be considered irrelevant when combined with PH features but also that PH features might lead the models astray. The results achieved by my augmented model shed light on the interaction between PH vectors, negation features, and LLMs. These complex interactions highlight potential challenges in optimizing their combined use for NLI.

4.4 The Verdict

The *RoBERTa_Neg* model will be integrated into my system to be used as the contradiction detection model. As discussed previously, my RoBERTa-based models are able to more accurately determine if two statements contradict (compared to the other models I previously examined). Combining the regular version of this LM with the negation word count feature allows the model to determine if two statements are contradictory rapidly and accurately.

Model	**Matched**	**Mismatched**
[Wu et al., 2022]	**85.4%**	**85.9%**
BBU	*72.68%*	*70.06%*
BBU_Neg	60.97%	33.13%
RoBERTaB	*80.36%*	*80.22%*
RoBERTaB_Neg	80.69%	80.24%

Table 4.4: Results achieved by each of my augmented models on M.Test and Mm.Test. The vanilla model results are shown in italics, and the baseline is presented at the top. The best results in each column are shown in bold.

Model	**Accuracy**	**F1-Score**	**Precision**	**Recall**
[Li et al., 2017]	84.27%	N/A	N/A	N/A
BBU	85.24%	0.8529	0.8560	0.8524
BBU_Neg	85.42%	0.8548	0.8576	0.8542
BBU_PH_MLP	14.63%	0.0377	0.3083	0.1463
BBU_PH_SVM	58.74%	0.4849	0.5773	0.5874
BBU_Neg_PH_MLP	28.70%	0.1280	0.0824	0.2870
BBU_Neg_PH_SVM	55.71%	0.5567	0.5564	0.5571
RoBERTaB	88.90%	0.8892	0.8908	0.8890
RoBERTaB_Neg	**89.01%**	**0.8903**	**0.8915**	**0.8901**
RoBERTaB_PH_MLP	28.70%	0.1280	0.0824	0.2870
RoBERTaB_PH_SVM	56.69%	0.4102	0.3213	0.5669
RoBERTaB_Neg_PH_MLP	28.70%	0.1280	0.0824	0.2870
RoBERTaB_Neg_PH_SVM	55.23%	0.5524	0.5527	0.5523

Table 4.5: The results achieved by each of my models on the SICK dataset. The results from the vanilla models are shown in italics, and the baseline is shown at the top. The best results in each column are shown in bold.

Model	**Accuracy**	**F1-Score**	**Precision**	**Recall**
Wang et al. [2021]	93.1%	N/A	N/A	N/A
BBU	*92.63%*	*0.9263*	*0.9263*	*0.9264*
BBU_Neg	92.36%	0.9240	0.9248	0.9236
RoBERTaB	*94.27%*	*0.9427*	*0.9432*	*0.9427*
RoBERTaB_Neg	**94.31%**	**0.9432**	**0.9434**	**0.9431**

Table 4.6: Results achieved by each of my models on the SNLI dataset. The vanilla model results are shown in italics, and the baseline is presented at the top. The best results in each column are shown in bold.

Chapter 5

Quality Control: Evaluation

With the different parts mentioned in previous Chapters, how should they be jigsawed together to present the best possible dialogue system for video game NPCs?

To determine how best to combine the pieces discussed in previous Chapters, a series of quantitative tests can be performed. This Chapter analyses the time it takes for ChatNPC to reply to a prompt, how much data (measured in megabytes) is sent to both the OpenAI and Pinecone API, and how much data is received from said APIs (also measured in megabytes).

To test these three quantitative metrics in ChatNPC, experiments were conducted involving each iteration of the fact rephrasal system (having the system active or not) paired with each iteration of the retrieval metric (cosine similarity and impact score).

The experiment consists of the same three questions — regardless of the agent's reply — being proposed, in order, to each of the NPCs five times. The questions are displayed in Table 5.1. Throughout this conversation, the number of megabytes sent, number of megabytes received, and the time to reply are kept track of, and after the 35th conversation (five conversations $\times$ seven characters) these values are averaged and

used in the following analysis.

Question	Description
1	What do you do?
2	What's your favorite drink to make? What're the secret ingredients?
3	Does your favorite drink to make pair nicely with any dishes on your menu?

Table 5.1: The questions using in quantitative experiments of ChatNPC

5.1 Information Sent and Received

As ChatNPC leverages the OpenAI and Pinecone APIs, both of which rely on network communication to send and receive data, monitoring the amount of data being transmitted is crucial in order to ensure ChatNPC operates reliably, efficiently, and in a cost-effective manner. As Tables 5.2 and 5.3 present, not using the fact rephrasal system and the cosine similarity retrieval method results in less data being sent over the network. This is expected, as for every phrase the system rephrases, the OpenAI API is called to query the language model, and the Pinecone API is called to save each rephrased fact to the knowledge base. For each call of the impact score retrieval method, the Pinecone knowledge base is also queried, further increasing the amount of data sent back and forth. Clearly, there are trade-offs between the different retrieval techniques and the use of the fact rephrasal system or not. Let's analyze how these different components affect the response time before coming to decision about what to include in the

final iteration of the system.

Retrieval method	No fact	Fact
Cosine Similarity	**0.86**	1.08
Impact Metric	0.92	2.57

Table 5.2: Average megabytes received per token during a conversation with ChatNPC. The lowest value is presented in bold.

Retrieval method	No fact	Fact
Cosine Similarity	**1.10**	2.00
Impact Metric	1.12	3.51

Table 5.3: Average megabytes sent MB per token during a conversation with ChatNPC. The lowest value is presented in bold.

5.2 Response Time

There is a metric which, as a video game enjoyer, I particularly value in my conversations with NPCs — response time. When I interact with a non-player character (NPC), it's usually for a particular purpose such as advancing a quest or gathering information about the area. The longer it takes to obtain the desired information, the more it feels

like my time isn't being respected. A timely reply from a character allows me to get the information required from that interaction and be on my way. Delaying that response can break immersion — preventing the player from being fully engrossed in the game.

Table 5.4 presents the average time (in seconds per token[1]) ChatNPC takes to respond to a user query, with the different retrieval methods and fact rephrasing systems in play. The combination of the impact metric and the absence of the fact rephrasing system offers, on average, the shortest time to reply. An interesting observation arises when the cosine similarity is paired with the fact rephrasal system, as even though the impact metric was, on average $0.0309s$ quicker to reply than the cosine similarity when the fact rephrasal system wasn't in play, coupling the impact metric with it caused ChatNPC to take longer to reply. Clearly, the fact rephrasal system causes ChatNPC to send more data over a network connection and take more time to reply. However, the absence of the fact rephrasing system leads the system to provide lower quality responses, often referencing information irrelevant to the query the user poses. This is most likely due to the context being retrieved from Pinecone being unedited sentences, often providing more information than is necessary to answer the query, confusing the LLM at the dialogue system's core.

5.3 Analysis

Whilst the dialogue system takes less time to reply when the fact rephrasing system isn't in use, the operating cost rises drastically (with more tokens being passed to the LLM as context) and the quality of the replies falls off. Not to spoil too much, the

[1]I know it's been a bit since we've talked about these, they're described in Section 2.1 if you need a refresher.

Retrieval method	No fact	Fact
Cosine Similarity	0.1405	0.3179
Impact Metric	**0.1096**	0.3889

Table 5.4: The time, in seconds, ChatNPC takes to reply per token in the provided reply. The lowest value is presented in bold.

final iteration of the ChatNPC system will be unveiled in the next chapter, but the impact score retrieval metric and fact rephrasal system will be included. Future work in regarding these components could be dedicated to improving the response time of the fact rephrasal system.

Chapter 6

From World 1 to 8: Current Pipeline

This chapter breaks down the entire dialogue system, along with providing an example of how the system operates at each step. Algorithm 1 outlines the workings of ChatNPC, with visual representation provided in Figure 6.1.

6.1 Username

We haven't spoken about usernames much, but they allow for users to access the system and have their agent replies compartmentalized. As Section 3.1.2 explains, the inputted username is used as a filter for the retrieval process — ensuring any previous interactions the user has had with ChatNPC can contribute knowledge to future conversations and that only a single knowledge base is required for the entire system. Figure 6.2 shows this landing page.

Algorithm 1 A high-level overview of the dialogue system.

Input: Choose character

Retrieve username email

Break character background information into facts

Store background information facts in knowledge base

Store setting facts in knowledge base

while *user wants to interact* **do**

    Query user (q)

    Embed query (q_e)

    Store q_e in knowledge base

    Query knowledge base with q_e and retrieve D_r

    for *each record in* D_r **do**

        Calculate *impact*

    end

    Rank all records by *impact*

    Find i_k by skimming the top k records off the top of the ranked records

    Use information within i_k to build a prompt

    Large language model processes the prompt and generates a reply (R)

    Split R into a collection of facts (R_f)

    for *each fact in* R_f **do**

        Store each fact in R_f independent of each other in knowledge base

    end

    Output response to user

end

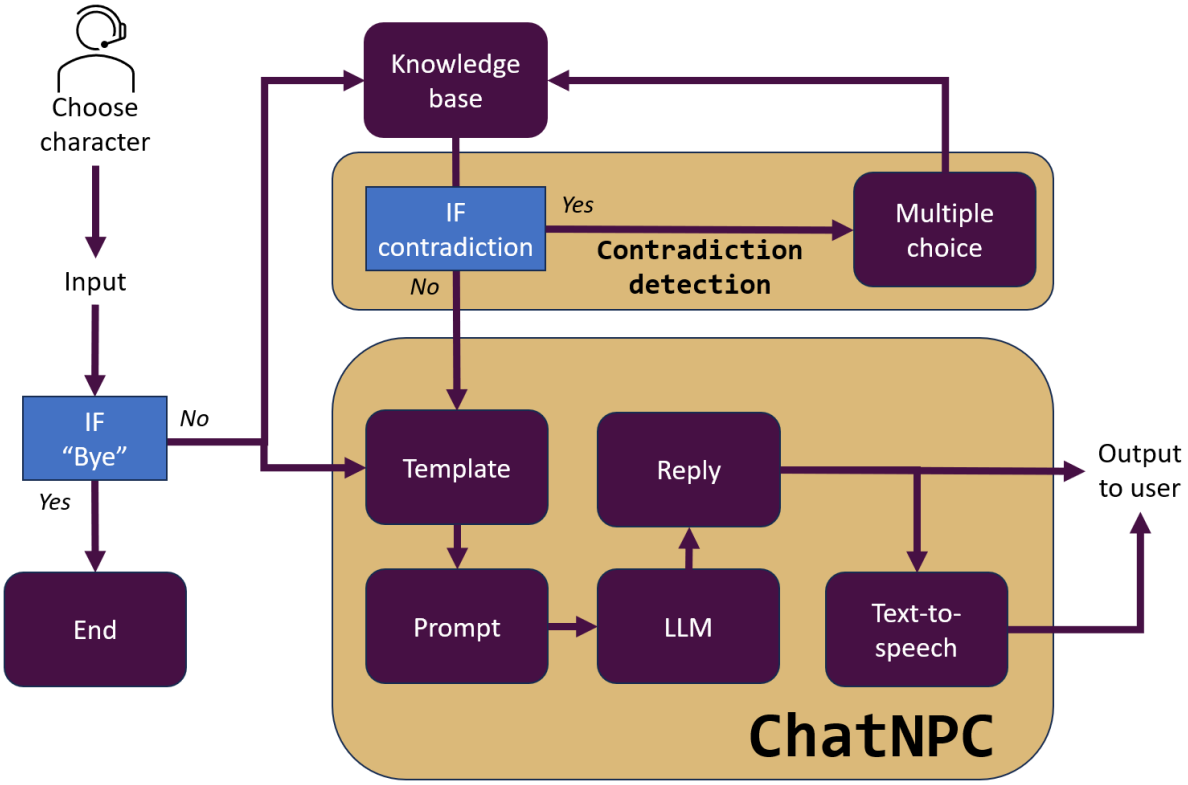


Figure 6.1: The flow chart of the ChatNPC system.



Figure 6.2: The landing page for ChatNPC, where a user inputs a username to enable isolated character memories and interactions.

6.2 Character Selection

Prior to any interaction, the character the user will be conversing with is identified. Through a sidebar on the website, the user can click on and select a character, activating the text box and allowing it to be used to send messages. If the selected character's history is not in the knowledge base, it is loaded after character selection using the procedure outlined in Section 3.1.2. The state of ChatNPC before a character is chosen is displayed in Figure 6.3. Figure 6.4 displays ChatNPC's visuals after a character is selected for conversation. For the purpose of this demonstration, we will converse with Melinda Deek.

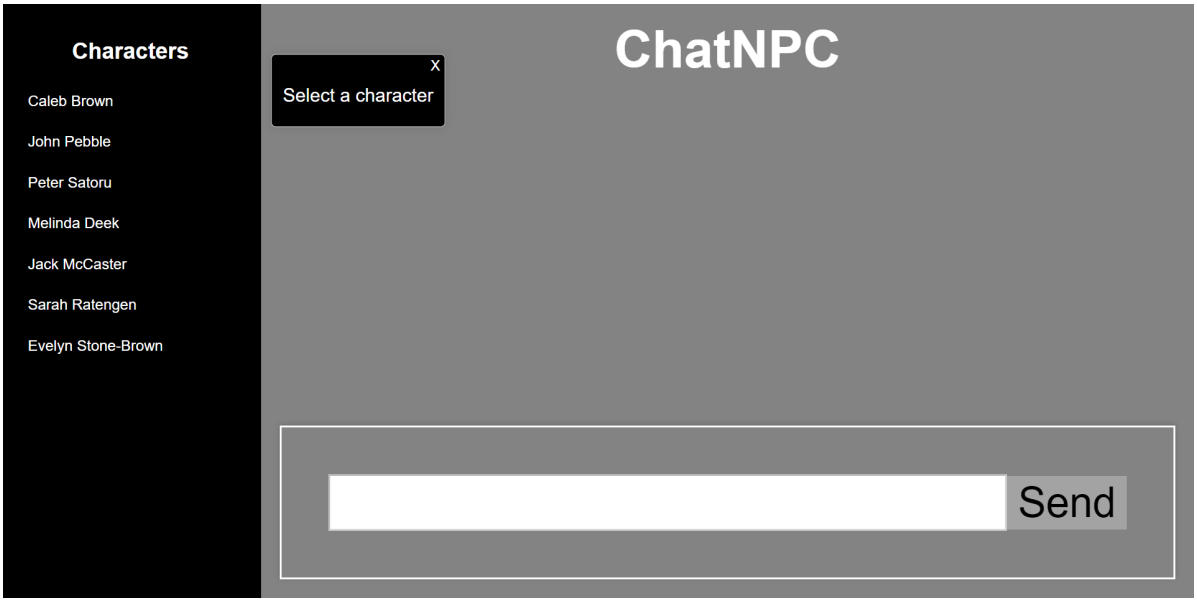


Figure 6.3: After selecting a username, ChatNPC brings you to the character selection screen.

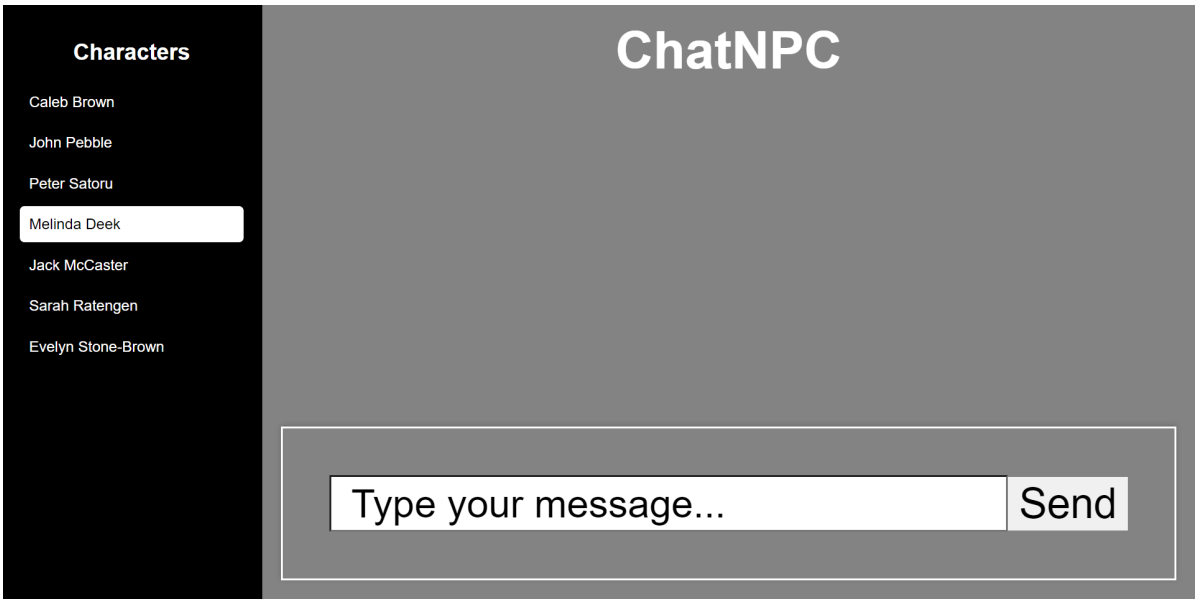


Figure 6.4: The state of ChatNPC after choosing a character to converse with. Note: you are able to type in the text box at the bottom of the screen, and the pop-up telling you to select a character has disappeared.

6.2.1 List of Characters

I created seven fictional characters and a fictional town to test my dialogue system. They vary in their history, relationships, and social status, and their names can be seen on the left hand side of the ChatNPC webpage. The backgrounds of each character can be found in the the Appendix.

6.3 User Query

The system kicks off a dialogue with the user by prompting the user to begin the conversation. The user's query is then embedded and passed to the database.

In our example, Melinda Deek is asked: "What do you do?"

6.4 Query Knowledge Base

With the user's embedded query in hand, the database retrieves all records stored in the character's partition of the database. These records are then sent from Pinecone, the external vector database, *back* to the dialogue system to have their impact scores calculated. Each record within i_k is then checked for contradictory information, and used in prompt engineering.

In the example we are following with character Melinda Deek, the knowledge based provided these facts as context to the aforementioned query:

- I take great pride in upholding the values of chivalry.
- I was trained from a young age in the ways of chivalry and combat.
- I come from a long line of knights.

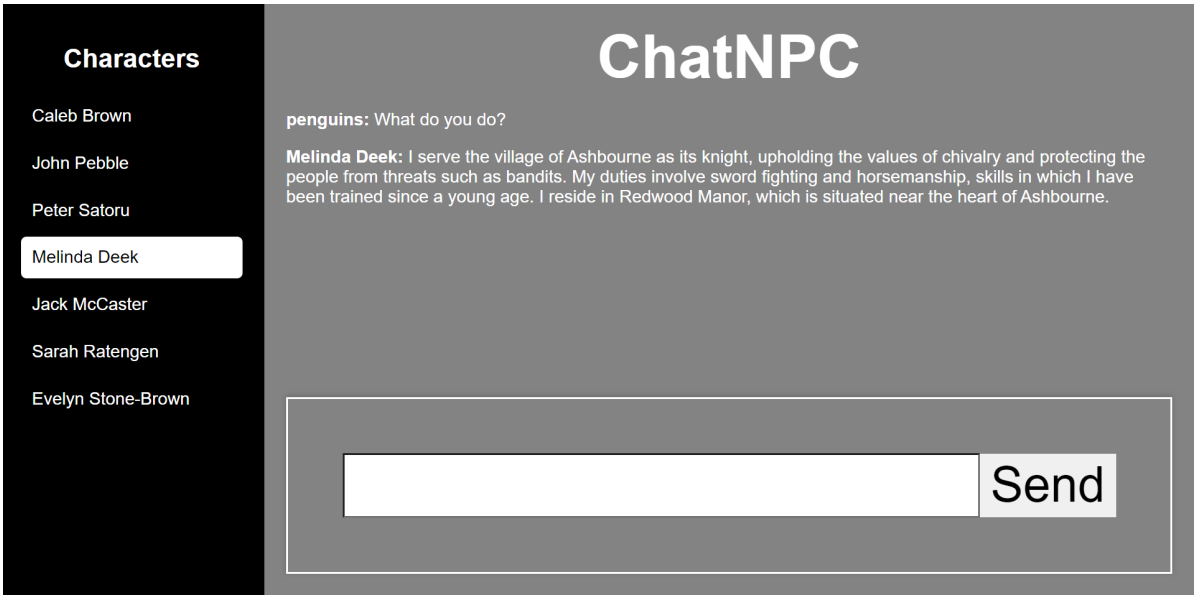


Figure 6.5: A dialogue between myself — username penguins — with Melinda Deek, a fictitious character created for ChatNPC. When asked "What do you do?", Melinda replied with "I serve the village of Ashbourne as its knight, upholding the values of chivalry and protecting the people from threats such as bandits. My duties involve sword fighting and horsemanship, skills in which I have been trained since a young age. I reside in Redwood Manor, which is situated near the heart of Ashbourne."

My initial query and the character's response can be seen in Figure 6.5.

6.5 Contradiction Detection

After querying the knowledge base, the contradiction detection system compares the sentence embedding of the query to the sentence embedding of each fact within i_k. If a contradiction is detected, the system outputs a message[1] to the user asking them to

[1]Select one of the following options to be identified as a "True" option. [Name of character you are conversing with]'s memory will be updated accordingly.

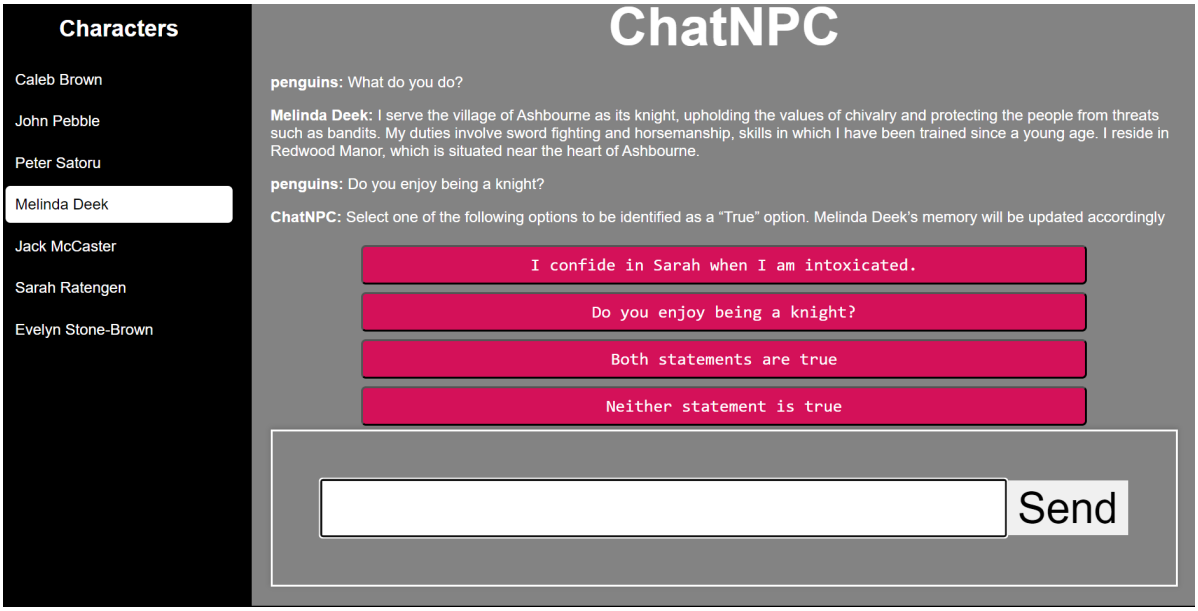


Figure 6.6: After selecting a username, ChatNPC brings you to the character selection screen.

choose one of four options to ensure the database is updated to only contain "true" facts. Each option has a different effect on the knowledge base, outlined in Table 6.1 and the multiple choice prompt can be seen in Figure **??**. After the user selects an option, the user's original prompt is using in prompt engineering, and the database is re-queried to provide up-to-date context.

6.6 Prompt Engineering

The system parses the records returned by the retrieval process and extracts the text from the metadata of each record in i_k.

When prompting an LLM the prompt has to be as explicit as possible, any ambiguity can lead to undesirable output. The prompt template found to be the most effective at the time of writing is found below, where $< 0 >$ is replaced with either poor, satisfactory,

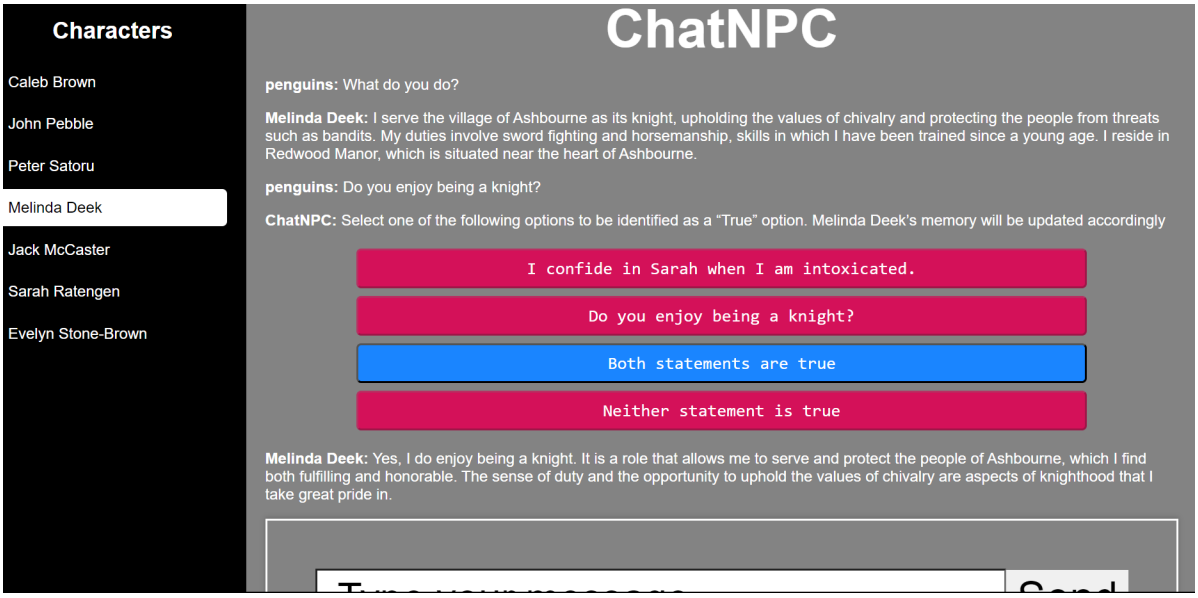


Figure 6.7: The selected option changes to a different colour, and Melinda then replies to our initial prompt with the updated context.

Selected Option	Knowledge base changes
First	None
Second	Remove first option, add second
"Both"	Add second option
"Neither"	Remove first option

Table 6.1: How the knowledge base is updated depending on the option chosen by the user.

or formal, $< 1 >$ is replaced with each record within i_k (in descending order of impact score) appearing on a new line, and $< 2 >$ rounds out the prompt by getting replaced with the user's query in natural language.

> Use $< 0 >$ grammar. Use first person. Do not mention that you are an AI language model, the user knows. Reply clearly based on the context. When told new information, reiterate it back to me. Do not mention your background, or the context unless asked, or that you are fictional. Do not provide facts you would deny.
>
> Context: $< 1 >$
>
> Question: $< 2 >$
>
> Answer:

Iterating over the structure of a prompt is an essential component of prompt engineering. Each part of the prompt exists to tackle a specific issue LLM replies to previous prompts exhibited. The breakdown of which issues each phrase solves is:

- *"Use poor/satisfactory/formal grammar."* — Characters in different societal classes are assumed to have different education levels. Asking the LLM to use a distinct type of grammar makes it more immersive for the user.
- *"Use first person."* — When not explicitly told this, the LLM would more often than not refer to the character it was emulating in the third person. Adding this section to the prompt causes the LLM to exclusively use first person.
- *"When told new information, reiterate it back to me."* — Due to the manner in which the data retrieval system filters through utterances throughout conversations

(ignoring those spoken by the user), the LLM is told to reiterate information it doesn't know back to the user in order to have that information stored for later reference.

- *"Do not mention your background or the context unless asked, or that you are fictional."* — Similar to the previous point about the LLM using third person, the LLM would occasionally be too transparent and refer to itself as a fictional character or information not relevant to the query being posed by the user.

6.6.1 Alternate Templates

The prompt template has undergone an iterative review process, where I had to be more explicit with each iteration. To save funds when first testing the system, as the cost of using `gpt-3.5-turbo` is token based, I told the LLM to only reply in one sentence. This had the undesirable effect of the LLM occasionally outputting run-on sentences with too much information, so this instruction was omitted in future iterations. The LLM would also occasionally refer to itself as a chatbot (definitely immersion-breaking) or refer to the character it should be emulating in the third person. Some of the templates I tested before landing on the one currently used by ChatNPC include, in order of earliest to more recent:

- "Answer the question as receiver, a job, based on the context below. Suggest another person you know who may have an answer, but only if you don't. Act exaggerated. Context:"

- "Answer as receiver, a job, based on context. Suggest a person you know who may have an answer, but only if you don't. Exaggerate. Context:"

- "Answer as receiver, a job, based on context. Suggest a person you know who may have an answer, but only if you don't. Don't mention anyone not mentioned in context. Exaggerate. One sentence only. Context:"
- "Reply in a succinct, single sentence based on the context. If you are told new information, repeat it in your own words. Use poor/satisfactory/formal grammar. Answer directly without adding unnecessary information. Context:"
- "Use satisfactory grammar. Use first person. Reply clearly based on the context. When told new information, reiterate it back to me. Do not mention your background or the context unless asked, or that you are fictional. Do not provide facts you would deny. Context:"

6.7 Language Model

The LLM at the core of this dialogue system is `gpt-3.5-turbo`[2]. Upon receiving the engineered prompt, the LLM interprets and responds to it. This process is fast and doesn't require much space, at the trade off of \$0.0005 per $1,000$ tokens sent to the OpenAI API.

6.7.1 Tested Language Models

While building the system, a wide array of language models were tested for their query answering capabilities. While not subject to any formal rigorous testing, I spent a few weeks qualitatively evaluating a variety of language models to gauge their ability to act as the core LLM to my dialogue system. Each model was prompted with

[2] `https://platform.openai.com/docs/models/gpt-3-5`

Briefly tell me about your favourite game.

For context, the response `gpt-3.5-turbo` gave to the previous prompt was:

I don't have personal preferences or the ability to play games, but I can certainly tell you about popular games or help answer questions related to gaming. If you have a specific game in mind or any gaming-related queries, feel free to ask!

GPT-J

GPT-J-6B [Wang and Komatsuzaki, 2021] was implemented through HuggingFace[3] and responded to the prompt with:

I like playing games.

Dolly

Also implemented through HuggingFace[4], *dolly-v1-6b* [Conover et al., 2023] (referred to as *Dolly*) is a fine-tuned open-source model derived from *GPT-J* and fine-tuned on the Stanford Alpaca[5] corpus. *Dolly* responded to the prompt by adding to it, then answering its own question!

and why it's your favourite. What makes it special, and why? I am a huge fan of board games, and I particularly enjoy Euro.

[3] `https://huggingface.co/EleutherAI/gpt-j-6b`
[4] `https://huggingface.co/databricks/dolly-v1-6b`
[5] `https://crfm.stanford.edu/2023/03/13/alpaca.html`

LLaMA

The open-source LLM *LLaMA* [Touvron et al., 2023], released by Meta, was competitive with larger LLMs (namely GPT-3 [Brown et al., 2020], Chinchilla-70B [Hoffmann et al., 2022], and PaLM-540B [Chowdhery et al., 2022]) on a variety of benchmarks when it was introduced. The model's response to the prompt can be seen below.

> *I'm just an AI, I don't have personal preferences or feelings, so I don't have a favourite game. However, I can provide information and strategies for various games if you would like! Is there a particular game you are interested in learning more about?*

While similar to the `gpt-3.5-turbo` output, this model is locked behind Meta providing access to it, and on top of that requires a whopping 129**GB** to store the 70 billion parameter variant (25GB for 13 billion and 13GB for 7 billion[6]).

GPT 4

While just as fast and space-friendly as `gpt-3.5-turbo`, `gpt-4` costs more ($0.01 per $1,000$ tokens) to use. Despite being tuned on more dialogue-specific tasks, I opted to use `gpt-3.5-turbo` to save money while experimenting.

6.8 Text-to-Speech

The reply given by the LLM is sent to the OpenAI `tts-1` text-to-speech model to generate the audio to match each reply. This model costs 15 USD for every one million

[6] `https://www.reddit.com/r/LocalLLaMA/comments/157d89h/llama_v2_model_sizes/`

tokens converted from text to speech. Each character is paired to a different voice option provided by the `tts-1` model, those pairings are as follows:

- **John Pebble**: Alloy
- **Caleb Brown**: Echo
- **Peter Satoru**: Onyx
- **Melinda Deek**: Shimmer
- **Jack McCaster**: Fable
- **Sarah Ratengen**: Nova
- **Evelyn Stone Brown**: Shimmer

As you can see, the "Shimmer" voice is used twice, as there are seven characters and six voices, only two of which sound feminine. Once the audio file is generated by `tts-1`, it is saved to the server hosting the website.

6.9 Output

The dialogue system outputs the character's reply to the user's question in natural language, which is embedded and stored in the database using the methods outlined in Section 3.1.2. The LLM-generated text is then displayed to the player — seen in Figures 6.5, 6.6, and 6.7 — alongside the most recently generated audio for the character they are speaking to being played when the text appears. The user is then able to enter further questions, or say "bye" if they wish to exit the conversation. Saying "bye" terminates

the conversation and outputs the amount of data (in megabytes) sent back and forth during the dialogue to the terminal on the server hosting the website.

Chapter 7

Curtain Call: Conclusion

ChatNPC is a dialogue system that incorporates a multitude of unique components to improve the inherent question-answering capabilities of a large language model. Adding an external database — akin to a second memory — to an LLM allows the model to impersonate fictitious characters and retain memories of said characters' previous conversations. A contradiction detection system adds a critical thinking aspect to the dialogue system, as LLMs historically tend to struggle with natural language inference tasks. Whilst this system may not achieve state-of-the-art performance on specific natural language inference benchmarks, the incorporated model still performs on par with or just below the performance of those models that perform the best on the natural language inference datasets used in testing. The fact rephrasal system is leveraged by ChatNPC to populate the second memory with standalone facts, removing ambiguity from the statements the contradiction detection system works with. A more robust information retrieval metric than cosine score, the impact score, is incorporated into my dialogue system, causing more relevant information to be retrieved from the knowledge base with each query proposed by the user. These components all come together in ChatNPC — a

dialogue system that allows characters and their conversations to be tailored to separate users.

7.1 Great Scott! Future Work

To elevate ChatNPC past its current state, potential avenues of improvements are outlined for the character ecosystem, performance, interface, and contradiction detection system.

7.1.1 Evolving Environment

Whilst the characters I created for ChatNPC know of each other, the mechanism by which characters' memories are saved only saves said memories to a single namespace — even if a character divulges information about another. This opens up the possibility of an evolving environment, where interactions between the user and character A that discuss character B allow for character B's memory to be updated with the statements character A makes. For example, if John Pebble mentions that Sarah Ratengen facilitates the black market trade in Ashbourne through her tavern, but that information isn't added to Sarah's background, then when the user discusses black market trading with Sarah she may feign ignorance as the information John Pebble provided is strictly tied to his "memory".

To approach this evolving environment, a mechanism to determine which character B is in question, determine if the information being presented contradicts the information in character B's background information, and respond accordingly — by either having character A indicate they are unsure about the information (if a contradiction is de-

tected) or or by adding the new information to character B's memory (if a contradiction isn't detected).

An alternate avenue to allow the environment to evolve is by adding additional characters. By giving users the option to add their own character by uploading a name and a background, even more variety can be introduced into the environment.

7.1.2 Performance Trade-Offs

My implementation of ChatNPC made results in certain drawbacks that can be improved upon in future iterations of the project. As mentioned in Chapter 5, reducing the reply time of the system will improve the immersion users feel when interacting with any given character. To this end, another LLM may be required, as querying the OpenAI API for each fact rephrasal and reply request presents a bottleneck for the system. Perhaps separating the fact rephrasal and response generating components of ChatNPC to different LLMs could improve the response time, and potentially improve the quality of said responses.

7.1.3 Elevating the Interface

To prevent users from influencing each other's conversations and conversation history, username authentication can be added to ChatNPC. This feature would require a password be created after a user enters their username for the first time.

Another improvement to the system as a whole is hosting it on a more permanent web-sever than simply on my home PC. Whilst attempts have been made towards this goal — leveraging Docker, Amazon Web Services, gunicorn, and other web-hosting technologies — each attempt has proven unsuccessful thus far. Exposing ChatNPC to more

users allows not only for stress tests of the system but for an elevation of the environment by allowing new users to create other characters.

7.1.4 Contradiction Detection System Improvements

I will extend the natural language inference work I've already completed by looking into nuanced linguistic features (such as determining the subject of each statement and/or analyzing the overlapping words) and trying to leverage them to improve my contradiction detection model.

Extra Nuts and Bolts: Appendix

7.1.5 Character Background Information

The following Sections detail the background information of each character available to converse with using ChatNPC at the time of writing. These excerpts are broken down into facts and stored in the vector database after the user first selects a character to provide the LLM with background context for how the character works. The excerpt describing Ashbourne — the town each character in ChatNPC lives in — is stored alongside each character's background information.

Caleb Brown

Caleb Brown is a friendly and hardworking baker who was born and raised in Quarlet. He learned his trade from his father, who owned a bakery in Quarlet, and now runs his own business in Ashbourne. Caleb is known for his delicious pies and pastries, which are very popular among the locals and travelers passing through the town. He is married to Evelyn Stone-Brown, the local blacksmith, and they live in a cozy house in Ashbourne. Caleb relies on Evelyn to fill in gaps in his knowledge regarding Ashbourne and the surrounding area. Caleb is a middle-class citizen who is well-liked by the people of Ashbourne, and he is always striving to improve his bakery's business. He is detail-oriented and has excellent customer service skills. Caleb has a few close friends in town, including Peter Satoru, an elderly archer. However, he also has an enemy, John Pebble, who steals his freshly baked buns.

Evelyn Stone-Brown

Evelyn Stone-Brown is a 38-year-old Anglo-Saxon female blacksmith from Ashbourne. Her father was also a blacksmith, and she learned the trade from him. Evelyn takes pride in her work and is known for crafting some of the finest weapons and armor in the land. Her physical appearance includes a height of 6'2" and a weight of 175 lbs, with black hair and brown and green eyes. She has skills in metalworking, swordsmithing, and armorsmithing, and her inventory includes a blacksmith hammer, tongs, an anvil, metal scraps, and finished weapons and armor. Evelyn's personality traits include being proud, determined, honest, and loyal. She has allegiances with the Ashbourne Blacksmiths Guild and the Kingdom of Albion. Evelyn has quests to craft a suit of armor for the king's army and forge a magical weapon for a powerful wizard. She has dialogue options such as repairing a sword, discussing the best metal for a blade, and teaching blacksmithing. Evelyn has friends such as a local barkeep, an enemy in a rival Ashbourne blacksmith, and a romantic interest in Caleb Brown. She lives in Ashbourne.

Jack McCaster

Jack McCaster is a fisherman who spends most of his days on the rivers on either side of Ashbourne. He comes from a family of fishermen and has been practicing the trade for as long as he can remember. Jack is a rugged man with sun-kissed skin, blue eyes, and short, curly brown hair. He stands at an average height and has a muscular build due to years of pulling in nets and hauling in fish. Jack has a friendly and outgoing personality and enjoys sharing stories and laughs with his fellow fishermen. He takes pride in his work and is always on the lookout for ways to improve his catch. Jack's skills include navigating the waters, fixing nets, and identifying different types of fish,

like the chromafluke in the town's western river, and the blackfins in the eastern one. He is a respected member of the fishing community.

John Pebble

John Pebble is a 28-year-old Caucasian male thief who lives near a village. He comes from a lower-class family and has a family history in the village, with his father being a blacksmith. However, John never had any interest in his father's trade and turned to thievery after being branded as a disgrace to his family. His physical appearance includes a height of 5'9" and a weight of 160 lbs with brown hair and blue eyes. John's skills include pickpocketing and lockpicking, which he uses to steal from villagers and loot their homes when they are away. He has a sneaky, cautious, and greedy personality, and he prefers to avoid fights and flee from danger quickly. John's allegiances lie with the thieves' guild, and he aspires to steal the mayor's prized possession. He will occasionally provide false information, even if he knows the correct information. He has a fierce rivalry with a rival thief, and is working to eliminate them permanently. He has a small inventory consisting of a lockpick set and stolen gold. His relationships include friends such as Sarah the tavern owner and Jack the fisherman. His enemies are Melinda Deek and Caleb Brown.

Melinda Deek

Melinda Deek is a knight who takes her duty to serve the village of Ashbourne. Trained from a young age in the ways of chivalry and combat, Melinda is a brave and honorable woman who comes from a long line of knights. Standing at a height of 6'2" and weighing 183 lbs, Melinda exudes strength and courage. Melinda's skills include sword fighting

and horsemanship, which she uses to protect the village from threats such as bandits. Melinda is very loyal to the village and her family, and she takes great pride in upholding the values of chivalry. Melinda is terrified of chickens but denies it to appear fearless. Melinda lives in Redwood Manor, a large house near the heart of Ashbourne. Melinda has close relationships with Peter Satoru, an elderly archer, and Sarah Ratengen, the local tavern owner. Melinda confides in Sarah when intoxicated.

Peter Satoru

Peter Satoru is a respected and experienced archer in the town of Ashbourne. He has lived there his entire life, and at 65 years old, he still possesses excellent archery skills and a wealth of knowledge about combat and strategy. Despite his age, Peter is patient, resilient, and wise, making him a valuable asset to the community. He spends much of his time training younger archers in the town, passing on his expertise to the next generation. Peter's family background includes a father who was also an archer, which suggests a long tradition of archery in his family. His social status is lower class, but his allegiances lie with the people of Ashbourne, whom he serves and protects. Peter has formed close relationships with Jack McCaster, a local fisherman, and Melinda Deek, a fellow knight in the village.

Sarah Ratengen

Sarah Ratengen is a friendly and compassionate tavern owner in Ashbourne, known for her warm hospitality and willingness to listen. She inherited the business from her father, who ran the tavern before her, and now offers a cozy atmosphere, good food, and drinks to both locals and travelers. Sarah has a middle-class social status and comes

from a long line of tavern owners. She is skilled in cooking and bartending and has a red hair color and brown eyes. Her tavern is well-stocked with barrels of ale and wine, and she keeps food supplies in inventory. Sarah has friends among the locals, including Jack McCaster, a fisherman, Caleb Brown, the local baker, and Melinda Deek, a knight. Sarah has no known enemies or romantic interests, and she lives upstairs in the tavern.

7.1.6 Town Background Information

Ashbourne is a fiction town created for the fictional character of my fictional[1] world. The following excerpt was created to provide characters background about the town in which they live, but has no been incorporated into the information retrieval system yet.

Ashbourne

Ashbourne is a quaint and picturesque town located in the heart of a verdant valley, with two rivers that run parallel to each other, one on the West and one on the Eastern sides of town. The town's setting is stunning, with towering mountains in the background and rolling hills surrounding it. The lush forest to the north-east of the town provides the villagers with abundant resources, game, and firewood. The town's streets are lined with charming, half-timbered houses. The manicured gardens of the townsfolk are filled with vibrant flowers, the Azure Asters, Emerald Lilies, and Fire Poppies. The town square is the heart of the town, a bustling hub of activity, and a meeting place for the locals. The weekly Ashbourne market, held every Saturday, is a popular event where farmers from the surrounding countryside bring their wares, including fresh fruits, vegetables, meat, dairy products, and baked goods. The fish market is a unique feature of Ashbourne, as

[1] Okay we get it Ethan you made them all up.

the two rivers provide an abundance of fish. The fish are caught by local fishermen and are sold fresh at the market every day. The blacksmithy is located at the edge of town, and it is a hive of activity, with the blacksmith hammering away at red-hot metal to create weapons, tools, and other metal goods. The tavern, located in the center of town, is a popular spot for the locals to gather and socialize over a pint of ale or mead. The tavern has a cozy atmosphere, with a large fireplace, wooden beams, and comfortable chairs. The archery range is located just outside of town, and it is a popular spot for the villagers to practice their skills. The range is surrounded by tall trees and rolling hills, providing a perfect backdrop for archery practice. Recreational activities in Ashbourne are numerous, including fishing in the rivers, boating, hiking, and hunting in the forest. The village also has a church, located at the edge of town, serving as a spiritual center for the villagers.

Bibliography

Ulrich Bauer. Ripser: efficient computation of Vietoris-Rips persistence barcodes. *J. Appl. Comput. Topol.*, 5(3):391–423, 2021. ISSN 2367-1726. doi: 10.1007/s41468-021-00071-5. URL https://doi.org/10.1007/s41468-021-00071-5.

Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 632–642, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1075. URL https://aclanthology.org/D15-1075.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Tianqi Chen and Carlos Guestrin. XGBoost. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, aug 2016. doi: 10.1145/2939672.2939785. URL https://doi.org/10.1145%2F2939672.2939785.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022.

Mike Conover, Matt Hayes, Ankit Mathur, Xiangrui Meng, Jianwei Xie, Jun Wan, Ali Ghodsi, Patrick Wendell, and Matei Zaharia. Hello dolly: Democratizing the magic of chatgpt with open models, 2023. URL `https://www.databricks.com/blog/2023/03/24/hello-dolly-democratizing-magic-chatgpt-open-models.html`.

Leyang Cui, Yu Wu, Jian Liu, Sen Yang, and Yue Zhang. Template-based named entity recognition using bart, 2021.

Raj Dabre, Chenhui Chu, and Anoop Kunchukuttan. A comprehensive survey of multilingual neural machine translation, 2020.

Avisha Das, Salih Selek, Alia R Warner, Xu Zuo, Yan Hu, Vipina Kuttichi Keloth, Jianfu

Li, W Jim Zheng, and Hua Xu. Conversational bots for psychotherapy: a study of generative transformer models using domain-specific dialogues. In *Proceedings of the 21st Workshop on Biomedical Language Processing*, pages 285–297, 2022.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

Matthew Dunn, Levent Sagun, Mike Higgins, V. Ugur Guney, Volkan Cirik, and Kyunghyun Cho. Searchqa: A new qa dataset augmented with context from a search engine, 2017.

Wafaa S El-Kassas, Cherif R Salama, Ahmed A Rafea, and Hoda K Mohamed. Automatic text summarization: A comprehensive survey. *Expert systems with applications*, 165:113679, 2021.

Anthony Fader, Luke Zettlemoyer, and Oren Etzioni. Open question answering over curated and extracted knowledge bases. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1156–1165, 2014.

Peng Gao, Shijie Geng, Yu Qiao, Xiaogang Wang, Jifeng Dai, and Hongsheng Li. Scalable transformers for neural machine translation, 2021.

Cristina Garbacea and Qiaozhu Mei. Neural language generation: Formulation, methods, and evaluation, 2020.

Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. WARP: Word-level Adversarial ReProgramming. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4921–4933, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.381. URL https://aclanthology.org/2021.acl-long.381.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.

He-Liang Huang, Xi-Lin Wang, Peter P. Rohde, Yi-Han Luo, You-Wei Zhao, Chang Liu, Li Li, Nai-Le Liu, Chao-Yang Lu, and Jian-Wei Pan. Demonstration of topological data analysis on a quantum processor. *Optica*, 5(2):193, feb 2018. doi: 10.1364/optica.5.000193. URL https://doi.org/10.1364%2Foptica.5.000193.

Dan Jurafsky. Speech & language processing. Preprint on webpage at https://web.stanford.edu/~jurafsky/slp3/ed3book.pdf, January 2023.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Jimmy Lam, Willem-Paul Brinkman, and Merijn Bruijnes. Generative algorithms to improve mental health issue detection. 2021.

Quoc V. Le and Tomas Mikolov. Distributed representations of sentences and documents, 2014.

Luyang Li, Bing Qin, and Ting Liu. Contradiction detection with contradiction-specific word embedding. *Algorithms*, 10(2):59, 2017.

Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023a.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Yiyi Liu, Ruqing Zhang, Yixing Fan, Jiafeng Guo, and Xueqi Cheng. Prompt tuning with contradictory intentions for sarcasm recognition. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 328–339, 2023b.

Marco Marelli, Luisa Bentivogli, Marco Baroni, Raffaella Bernardi, Stefano Menini, and Roberto Zamparelli. SemEval-2014 task 1: Evaluation of compositional distributional semantic models on full sentences through semantic relatedness and textual entailment. In *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*, pages 1–8, Dublin, Ireland, August 2014. Association for Computational Linguistics. doi: 10.3115/v1/S14-2001. URL `https://aclanthology.org/S14-2001`.

Oren Melamud, Jacob Goldberger, and Ido Dagan. context2vec: Learning generic context embedding with bidirectional LSTM. In Stefan Riezler and Yoav Goldberg, editors, *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 51–61, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1006. URL `https://aclanthology.org/K16-1006`.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

Kostadin Mishev, Ana Gjorgjevikj, Irena Vodenska, Lubomir T Chitkushev, and Dimitar Trajanov. Evaluation of sentiment analysis in finance: from lexicons to transformers. *IEEE access*, 8:131662–131682, 2020.

OpenAI. Gpt-4 technical report, 2023.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text, 2016.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.

Laria Reynolds and Kyle McDonell. Prompt programming for large language models: Beyond the few-shot paradigm. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–7, 2021.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.

Ben Wang and Aran Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. `https://github.com/kingoflolz/mesh-transformer-jax`, May 2021.

Sinong Wang, Han Fang, Madian Khabsa, Hanzi Mao, and Hao Ma. Entailment as few-shot learner. *arXiv preprint arXiv:2104.14690*, 2021.

Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory, 2023.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners, 2022.

Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122. Association for Computational Linguistics, 2018. URL `http://aclweb.org/anthology/N18-1101`.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.emnlp-demos.6`.

Xiangcheng Wu, Xi Niu, and Ruhani Rahman. Topological analysis of contradictions in text. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2478–2483, 2022.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. Swag: A large-scale adversarial dataset for grounded commonsense inference, 2018.